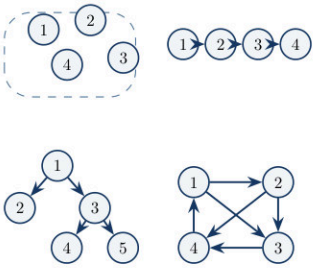
《数据结构与算法》（C++）　可视化专题讲义

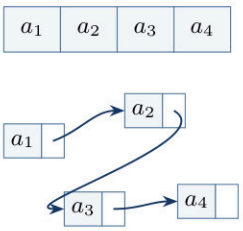
数据结构的基本概念与算法

从“一张成绩表”到“$O(\log_2 n)$”——第一次学也能看懂

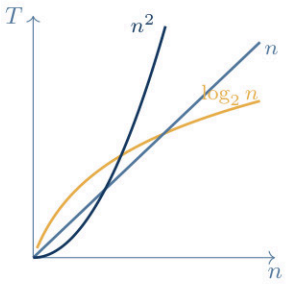
逻辑结构



存储结构



时间复杂度



适用对象　第一次学习本专题的大学本科生（大二上）

配套练习　38 道 = 基础 25 + 中等偏下 10 + 跨学科迁移 3

排版　XeLaTeX ·TikZ ·PGFPlots 全矢量图解

阅读说明

这份讲义由两页手写课堂笔记重构而来。笔记很“省”：它记下了结论，却省掉了“为什么”“怎么来的”“在哪里会错”。讲义要做的，就是把这些省掉的部分一步一步补回来。颜色有固定含义，看久了你会形成条件反射：

定义 / 核心公式 / 定理　为什么 / 直观 / 证明　例题 / 生活 / 历史　易错 / 原图纠错
总结 / 自检　相似知识对照　教学补全　✓可直接迁移 △加条件 ✗不可迁移

建议的读法：先通读第一章、第二章（每个知识点按“问题 → 图 → 定义 → 例子 → 易错 → 对照”展开），每节末尾用“学完这里，你应该能够回答”自测；再独立完成第三章的 38 道练习，最后对照第四章的答案与图解。

目录

阅读说明 2

本专题学什么与课程位置 3

学科小史 3

从课本到现实 4

原始资料重新编码与纠错 4

知识点总览与关系图 5

第一章 数据结构的基本概念 6
1.1 K1 数据、数据元素、数据项（Data, Data Element, Data Item） . . . 6
1.2 K2 数据结构与二元组 (D, R) . . . 8
1.3 K3 数据类型与抽象数据类型（Data Type, ADT） . . . 10
1.4 K4 逻辑结构：集合、线性、树形、图形 . . . 12
1.5 K5 存储结构：顺序、链式、索引、散列 . . . 14

第二章 算法与算法分析 17
2.1 K6 算法的定义与五个特性（Algorithm） . . . 18
2.2 K7 好算法的标准 . . . 20
2.3 K8 时间复杂度（Time Complexity） . . . 22
2.4 K9 空间复杂度（Space Complexity，教学补全） . . . 27

第三章 正式练习（38 题） 29
3.1 A. 基础巩固训练（25 题） . . . 29
3.2 B. 中等偏下提升训练（10 题） . . . 32
3.3 C. 跨学科迁移训练（3 题） . . . 33

第四章 答案与图解 34
4.1 基础巩固·详细解析 . . . 34
4.2 中等偏下提升·详细解析 . . . 40
4.3 跨学科迁移·详细解析 . . . 43

第五章 总结与延伸 45

本专题学什么？它在课程里处在什么位置？

《数据结构与算法》整门课要回答两个问题：**数据怎么组织**（数据结构），以及**问题怎么一步步解决**（算法）。本讲是全课的“词典”：后面每一章——线性表、栈与队列、树、图、查找、排序——都会反复使用这里定义的词：数据元素、逻辑结构、存储结构、算法、时间复杂度。如果这些词含糊，后面每一章都会含糊。

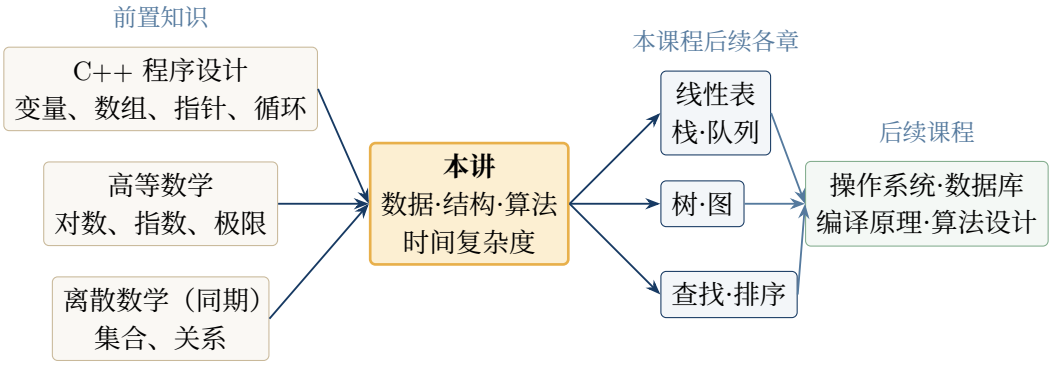


图 1：本专题在课程体系中的位置：左边是你已经学过或正在学的，右边是马上要用到它的

本讲回答五个问题： ①计算机里的“数据”分几层？ ②“数据结构”为什么能写成 (D, R)？ ③同一份数据，为什么有“逻辑上怎么看”和“内存里怎么放”两种结构？ ④什么样的步骤才配叫“算法”？ ⑤怎样不运行程序就比较两个算法谁快——即 $O(\log_2 n)$ 这类记号到底是什么意思？

> **前置知识速查**　本讲只用到三样旧知识，不熟的话先看这里：**(1) 对数**：$2^x = n \iff x = \log_2 n$，例如 $\log_2 8 = 3$，$\log_2 1024 = 10$；记号 $\lfloor x \rfloor$ 表示“不超过 x 的最大整数”，如 $\lfloor \log_2 10 \rfloor = 3$。**(2) 集合与有序对**：$\{a, b\}$ 是集合，元素无先后；$\langle a, b \rangle$ 是有序对，a 在前、b 在后。**(3) C++ 循环**：`while(条件) 语句`；先判断条件，条件成立才执行语句，再回头判断。

学科小史：这些概念是怎样出现的？

早期的计算机程序员面对的是一个很朴素的困难：机器只认识一格一格的内存，而人要处理的是名单、账本、家谱、线路这类“有关系的数据”。怎样把“关系”放进只有地址的内存？怎样判断一种做法比另一种好？数据结构与算法分析，就是在回答这两个问题的过程中逐渐成形的。

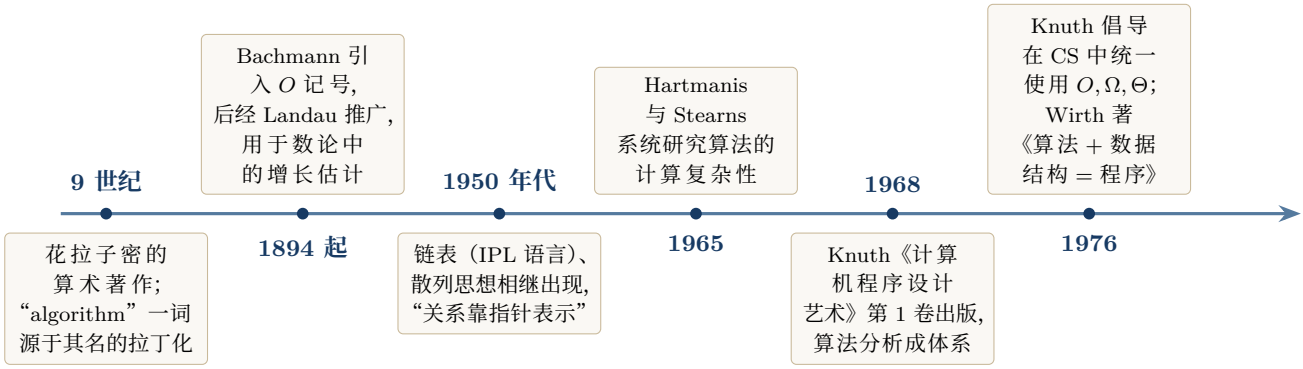


图 2：从“计算步骤”到“算法分析”的几个关键节点（年代采用通行说法，个别节点的“首创”归属学界说法不一，本图只标示代表性工作）

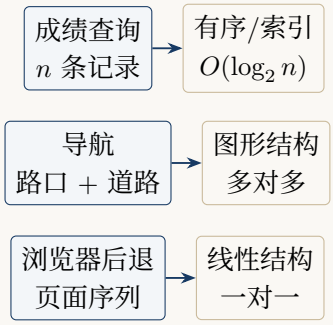
这条时间轴想说明一件事：**“数据 + 关系”与“步骤 + 代价”是两条并行的线**。前一条线最终变成了本讲的“逻辑结构 / 存储结构”，后一条线变成了“算法的特性 / 时间复杂度”。1976 年 Wirth 的书名“算法 + 数据结构 = 程序”恰好把两条线合在了一起——这也是这门课名字的由来。今天我们

之所以仍要学它，是因为无论硬件怎样变快，“n 变大一倍时代价增加多少”这个问题从来没有过时：硬件快 10 倍，挡不住 n^2 的算法在 n 增大 10 倍时慢 100 倍。

从课本到现实：这个知识到底有什么用？

三个你每天都在用的“数据结构 + 算法”

①教务系统里查成绩。全校几万条学生记录，每条记录就是一个“数据元素”。系统按学号建立有序或索引结构，查找时每比较一次就排除一半，几万条记录十几次比较就能定位——这正是本讲例题中 $\log_2 n$ 的来源（$\log_2 50000 \approx 15.6$）。
②地图导航。路口是数据元素，道路是“关系”，一张城市地图就是一个**图形结构**；导航软件做的事是在这张图上找最短路径（第 6 章会学）。
③浏览器“后退”。你访问过的网页按先后排成一条线（**线性结构**），后退总是回到“最后访问的那个”——这就是后面要学的“栈”。



原始资料重新编码与纠错

笔记里到底写了什么

下表按笔记原顺序列出可靠识别的内容（讲义正文会按学习逻辑重新排序）。

页	原笔记条目	识别结果（要点）
529	一、1. 数据	所有能输入到计算机中的描述客观事物的符号：文本、声音、图像、符号等
529	2. 数据项	有独立含义的数据最小单位，也称域；若干个数据项构成一个数据元素；数据项是不可分割的最小单位
529	3. 数据元素	数据的基本单位；别称：节点、记录；例：学号 100872、姓名 刘木、班级 1442、成绩 92 整行为数据元素，“92”为数据项
529	4. 例	“数据的逻辑结构从形式上可用三元组 (D, R) 表示，其中 R 是 () 的有限集”，选项 A 算法 B 数据元素 C 数据线 D 数据关系；红笔：D：收集数据，R：数据之间的关系
529	5. 数据类型	“数据的不可分割的单位是数据类型”
529	6. 逻辑结构和存储结构	逻辑结构：集合、线性结构、树形结构、图形结构；线性 $1 \to 2 \to 3$，树形一对多，图形多对多
530	②存储结构	顺序存储、链式存储、散列存储、索引存储；红笔：常用两种为顺序、链式
530	二、1. 概念	算法是对特定问题求解步骤的一种描述
530	2. 如何判断优劣	以算法执行所耗费的时间与占用空间来判断
530	3. 特点	有穷性、确定性、可行性、输入（0 或多个）、输出（1 或多个）
530	4. 好算法	正确性、易读性、健壮性、高效性、低存储性
530	5. 时间复杂度	一般将算法基本运算执行次数作为衡量标准；例：`t=1;` `while(t<=n) t=t*2; return t;`，$2^x = n$，$x = \log_2 n$，$O(\log_2 n)$

无法可靠辨认 / 需说明之处：(1) 例题选项 C 原图写作“数据线”，常见教材习题此处为“数据操作”，

以下属于推测，不属于原图可靠识别结果：可能是笔误；它不影响答案 D。(2) 原图有两处涂改带覆盖（“如何判断算法优劣?”之后、“1442”附近），被覆盖的原文无法恢复，已识别的可见文字不受影响。

原图纠错

原图纠错① 数据类型的定义

原始写法：“数据的不可分割的单位是数据类型。”
问题：“不可分割的最小单位”是**数据项**（笔记第 2 条已写过）。数据类型根本不是“单位”，而是一个“值的集合 + 操作”的整体。
规范写法：数据类型是**一组性质相同的值的集合，以及定义在这个值集合上的一组操作的总称**。例如 C++ 的 `int`：值集合是一段整数，操作是 $+, -, \times, /, \%$ 等。
影响：若按原写法记忆，会把“数据项”和“数据类型”混为一谈，选择题极易错。正文第 1.4 节专门讲。

原图纠错② “三元组”与“D 的含义”

原始写法：“可用三元组 (D, R) 表示”；红笔“D：收集数据”。
问题：(D, R) 只有两个分量，应为**二元组**；“收集数据”不是规范说法。
规范写法：数据结构可形式化为二元组 (D, R)，其中 D 是**数据元素的有限集**，R 是 D 上**关系的有限集**。答案 D（数据关系）正确。
为什么改：把 D 说成“收集数据”会丢掉“有限集”和“元素”两个关键词，而它们恰好是选择题的考点。

原图纠错③ 有穷性与时间复杂度两处表述不严谨

有穷性：原写“总是在执行若干次后结束”。规范说法是：**算法必须在执行有穷步之后结束，且每一步都可在有穷时间内完成**——“每一步有穷时间”这一半被省略了。
时间复杂度例题：原写 $2^x = n \Rightarrow x = \log_2 n$。这是“量级估计”，不是精确次数：精确地说，循环体执行 $\lfloor \log_2 n \rfloor + 1$ 次 $(n \geq 1)$。例如 $n = 10$ 时执行 4 次而不是 $\log_2 10 \approx 3.32$ 次。结论 $O(\log_2 n)$ 仍然正确，第 2.4 节给出完整证明。
用词：“易读性”通常称“可读性”；“高效性、低存储性”通常合称“高效率与低存储量需求”。属于用词习惯，不影响理解。

知识点总览与关系图

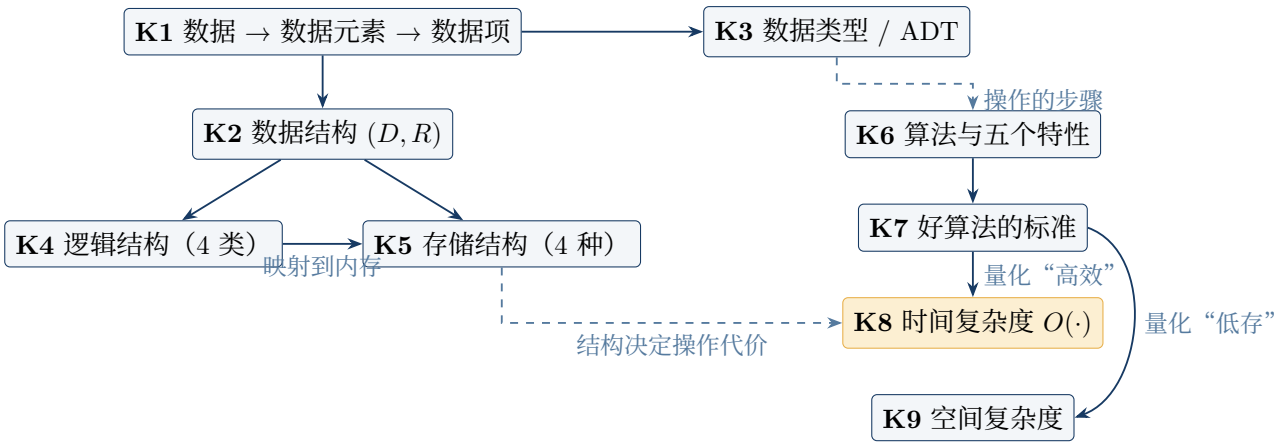


图 3: 本讲 9 个核心知识点及其关系。实线：逻辑先后；虚线：两条主线（结构、算法）之间的联系

第一章　数据结构的基本概念

本章回答三个问题：①计算机眼中的“一张成绩表”分成几层？②“数据之间的关系”怎样用数学写下来？③同一份数据，“逻辑上怎么看”与“内存里怎么放”为什么是两回事？

本章知识路线：数据 → 数据元素 → 数据项 → 数据结构 (D, R) → 数据类型 → 逻辑结构 → 存储结构。

1.1　K1 数据、数据元素、数据项（Data, Data Element, Data Item）

学科坐标：《数据结构与算法》第 1 章；前置：C++ 的变量与结构体；后续：线性表的“表项”、查找中的“关键字”、数据库中的“记录 / 字段”。

它解决什么问题

老师在笔记里举了一张成绩表。你一眼就能看出“刘木考了 92 分”，但计算机看到的只是一堆符号。要让计算机处理它，我们首先得约定：**哪一部分算“一个整体”，哪一部分算“不能再拆的零件”**。没有这个约定，就没法说“删除一个学生”“修改一门成绩”到底动的是哪一块。

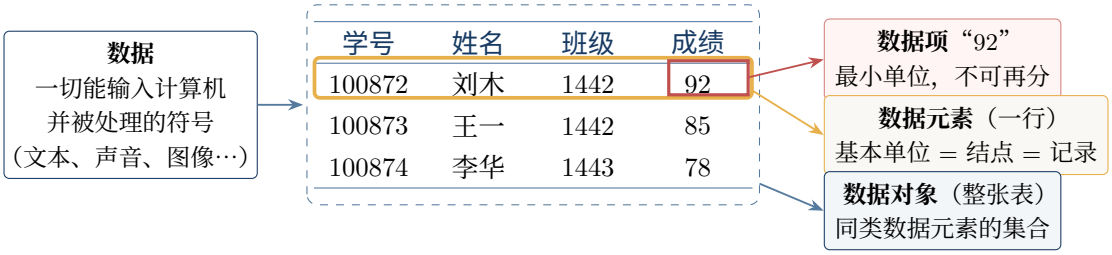


图 4：同一张成绩表的四个层次：红框是数据项，黄框是数据元素，虚线框是数据对象

图 4 从外到内读：最外层“数据”是一切符号的总称；同一类记录放在一起组成“数据对象”；每一行是一个“数据元素”，程序通常把它作为一个整体来增、删、查；每一格是一个“数据项”，它是有独立含义的最小单位。图中最应该看出的是：**“基本单位”是行，“最小单位”是格**。

定义 （K1）

数据：所有能输入到计算机中并被计算机程序处理的、描述客观事物的符号的总称，如文本、声音、图像、数值等。
数据元素：数据的**基本单位**，在程序中通常作为一个整体进行考虑和处理；也称**结点、记录**。
数据项：组成数据元素的、有独立含义的、**不可分割的最小单位**，也称**域（字段）**。一个数据元素可由若干数据项组成。
数据对象（教学补全）：性质相同的数据元素的集合，是数据的一个子集。例如“全体学生记录”。

> **一句话直观理解**　数据元素是“一个人的整条记录”，数据项是“这条记录里的一个格子”。删学生，删的是一整行；改成绩，改的是一个格子。

为什么这样分层？因为程序做的事情恰好有两种粒度：“对整个对象动手”（插入、删除、查找一个学生）和“对某一属性动手”（按成绩排序、改班级）。前者以数据元素为单位，后者以数据项为单位。笔记中“若干个数据项构成一个数据元素”说的正是这一层包含关系。

正例与反例

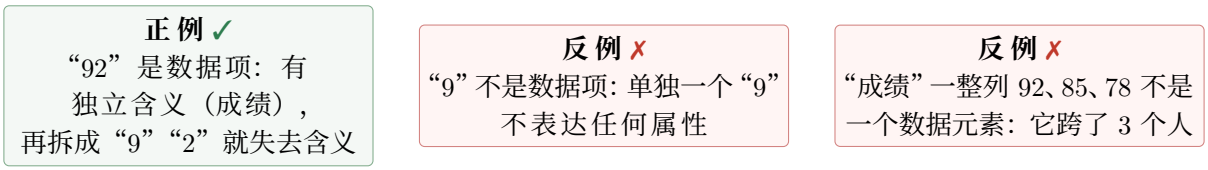


图 5：判断“是不是数据项 / 数据元素”的三个对照

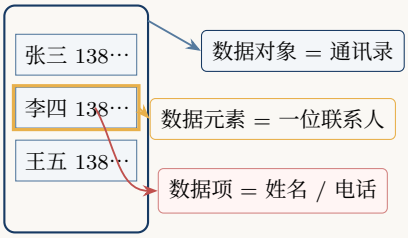
判断准则只有一句：**数据项看“含义是否独立且不可再分”，数据元素看“是否描述同一个对象的整体”**。

例题 1.1 数一数

已知：图 4 中的成绩表有 3 名学生，每名学生有 4 个属性。**求：**数据元素个数与数据项总数。
思路：一行 = 一个数据元素；一格 = 一个数据项。
第一步：行数 = 3，故数据元素有 3 个。**第二步：**每行 4 格，数据项共 $3 \times 4 = 12$ 个。
检查：表头“学号、姓名…”不是数据项的“值”，它们是数据项的名称，不计入。
本题真正想说明：数据项属于某个数据元素，“数据项总数 = 元素个数 × 每个元素的项数”。

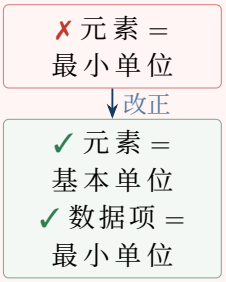
生活中的它　手机通讯录

场景：你手机里的通讯录。
对应：整个通讯录 ↔ 数据对象；一个联系人 ↔ 数据元素（记录）；“姓名”“电话”“备注” ↔ 数据项。
小估算：200 个联系人、每人 3 个字段，共 $200 \times 3 = 600$ 个数据项；删除一个联系人，一次删掉 3 个数据项。
边界：真实通讯录里一个人可以有好几个电话，字段数量不固定，这时“电话”本身是一组数据项（教材称组合项），比课本模型复杂。



易错点

错误想法：“数据元素是数据的最小单位。”
错在哪里：把“基本单位”和“最小单位”说反了。
正确想法：数据元素是基本单位（处理时的整体），数据项是最小单位（不可再分）。记忆口诀：**“元素是基本，数据项最小”**。



对照：数据元素 / 数据项 ↔ C++ 结构体变量 / 成员（思想重现型）

一句话定位：C++ 中用 `struct` 描述“一个数据元素长什么样”，用成员变量描述“它有哪些数据项”。

维度	数据结构课	C++ 程序
整体	数据元素	`struct` 变量
零件	数据项	成员 `.score`
同类整体的集合	数据对象	数组 `Stu a[3]`
访问零件	“某元素的成绩”	`a[0].score`

```
struct Stu { // 一个数据元素的样子
  long id; string name;
  int cls; int score; // 4 个数据项
};
Stu a[3]; // 数据对象
```

相同：“整体由若干零件组成，零件有名字和含义”。**不同及根源：**数据结构课讲的是概念，不关心语言；C++ 的成员还可以是数组或另一个结构体（可再分），所以“C++ 成员”不一定等于“不可再分的数据项”——根源在于语言允许嵌套。

旧知识（C++）中的结论	迁移	理由或反例
一个 `struct` 变量 = 一个数据元素	✓	都表示“同一对象的整体”
每个成员都是不可分割的数据项	△	仅当成员是基本类型时成立；成员是 `string` 或数组时可再分
`sizeof` 等于各成员大小之和	✗	编译器会做内存对齐，可能多出填充字节

学完这里，你应该能够回答

①数据元素和数据项哪个是“基本单位”，哪个是“最小单位”？②数据元素的两个别称？③ 3 条记录、每条 5 个字段，共几个数据项？④“成绩”这一整列为什么不是数据元素？

1.2　K2 数据结构与二元组 (D, R)

学科坐标：第 1 章；前置：K1、集合与有序对；后续：每一种具体结构（线性表、树、图）都是给 R 加不同限制得到的；相关课程：离散数学“关系”。

它解决什么问题

只有一堆数据元素还不够。同样三个学生，“按学号排成一队”和“按宿舍分组”是两种完全不同的组织方式。我们需要一种不依赖语言、不依赖画图的方式，把“元素之间的关系”精确写下来——这就是二元组 (D, R)。

定义 （K2）

数据结构：相互之间存在一种或多种特定关系的数据元素的集合。形式化地，数据结构是一个**二元组**

$$\boxed{\text{Data_Structure} = (D, R)}$$

其中 D 是**数据元素的有限集**，R 是 D 上**关系的有限集**。最常见的情形只有一个关系，它是由若干**有序对** $\langle a, b\rangle$ 组成的集合，$\langle a, b\rangle$ 表示“a 是 b 的直接前驱，b 是 a 的直接后继”。

图 6 上半部分逐项标出了两个分量的含义；下半部分说明了它们的几何对应：**D 中每个元素画成一个点，R 中每个有序对 $\langle a, b\rangle$ 画成一条从 a 指向 b 的箭头**。以后看到任何 (D, R)，都可以先“画点、再连箭头”，结构类型一眼可见。

为什么用有序对而不是集合 $\{a, b\}$？因为很多关系有方向：“a 排在 b 前面”与“b 排在 a 前面”是两件事。集合 $\{a, b\} = \{b, a\}$ 记不住方向，有序对 $\langle a, b\rangle \neq \langle b, a\rangle$（当 $a \neq b$）可以。

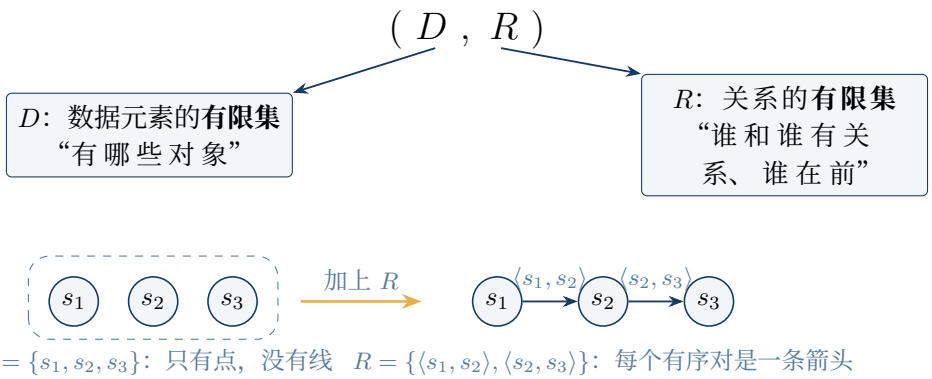


图 6: 标注公式图：D 给出“点”，R 给出“箭头”。三个学生按学号排队，就是给 D 加上两条有序对

例题 1.2 笔记原题：R 是什么的有限集？

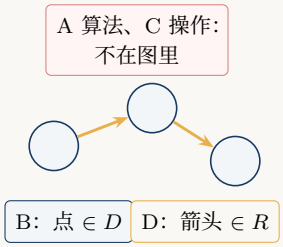
题目： 数据的逻辑结构从形式上可用二元组 (D, R) 表示，其中 R 是（　）的有限集。

A. 算法　B. 数据元素　C. 数据操作　D. 数据关系

思路： 对照定义：D 管“点”，R 管“箭头”。

逐项： A 算法是“步骤”，不在结构里；B 数据元素是 D 的成员；C 操作属于数据类型 / ADT；D 正确。**答案 D。**

本题真正想说明： (D, R) 只描述“有什么”和“什么关系”，不描述“怎么操作”。



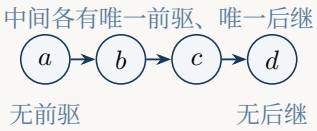
例题 1.3 由 (D, R) 画出结构

已知： $D = \{a, b, c, d\}$，$R = \{\langle a, b\rangle, \langle b, c\rangle, \langle c, d\rangle\}$。**求：** 画出结构并说出类型。

第一步： 画 4 个点。**第二步：** 按 3 个有序对连箭头 $a \to b \to c \to d$。

第三步： 检查：a 没有前驱、d 没有后继，其余各有唯一前驱与唯一后继 $\Rightarrow$ **线性结构**。

检查： 箭头数 3 = 有序对个数 3。



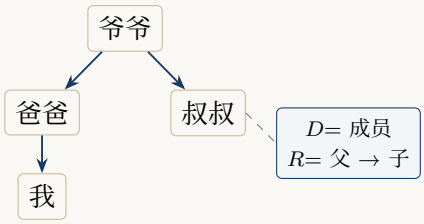
生活中的它　家谱

场景： 一本家谱记录“谁是谁的父亲”。

对应： 家族成员 $\leftrightarrow$ D；“父亲 $\to$ 子女” $\leftrightarrow$ 有序对 $\langle$ 父, 子 $\rangle \in R$。

小判断： $\langle$ 爷爷, 爸爸 $\rangle$ 与 $\langle$ 爸爸, 爷爷 $\rangle$ 不能互换——方向错了辈分就乱了，这就是要用有序对的原因。

边界： 真实家族还有婚姻、收养、母系等多种关系，要用多个关系 $R_1, R_2, \ldots$ 才能描述，这正是定义里说“关系的有限集”的原因。



易错点

错误想法： “$R = \{\langle a, b\rangle\}$ 和 $R = \{\langle b, a\rangle\}$ 是同一个结构。”

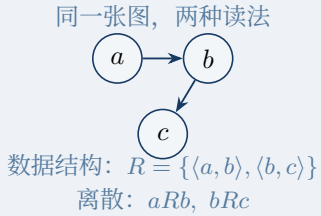
错在哪里： 有序对有方向，前者 a 是前驱，后者 b 是前驱，画出来箭头方向相反。　**正确想法：** 看

到 $\langle x, y\rangle$，就画 $x \to y$。　 

对照：(D, R) ↔ 离散数学中的“集合上的二元关系”与有向图 $G = (V, E)$（推广型）

一句话定位： (D, R) 中的单个关系，就是离散数学里“D 上的二元关系 $R \subseteq D \times D$”；画出来就是一张有向图。

维度	数据结构 (D, R)	离散数学 / 图论
对象	数据元素集 D	集合 A / 顶点集 V
关系	有序对集 R	$R \subseteq A \times A$ / 边集 E
关心什么	结构类型、怎样存、怎样操作	关系的性质（自反、对称、传递…）
有几个关系	可以有多个	通常讨论一个



相同：有序对的写法、“画点连箭头”的图示、集合运算规则完全一致。**不同及根源：**离散数学研究关系的性质，数据结构研究关系的形状（线性、树、图）以及它在内存中的实现——两门课的目的不同。

离散数学中的结论	迁移	理由或反例
R 是 $D \times D$ 的子集	✓	定义相同
关系可用关系矩阵表示	✓	数据结构中图的“邻接矩阵”就是关系矩阵（第 6 章）
关系的传递闭包有意义	△	对线性结构，R 本身只含“直接前驱”，不含“间接前驱”，求闭包后才得到“在前面”的关系

学完这里，你应该能够回答

① (D, R) 是几元组？D、R 各是什么的有限集？② $\langle a, b\rangle$ 画成哪个方向的箭头？③给一个 (D, R)，你能先画点再连线吗？

1.3　K3 数据类型与抽象数据类型（Data Type, ADT）

学科坐标：第 1 章；前置：C++ 的 `int`、`double`、`class`；后续：每一章开头都会给出该结构的 ADT（如“线性表的 ADT”）。笔记原写法有误，见“原图纠错①”。

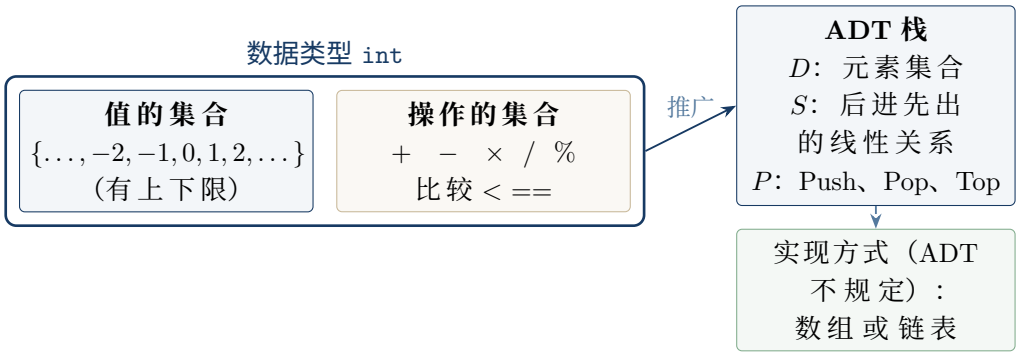


图 7: 左：数据类型 = 值集合 + 操作集合（缺一不可）；右：ADT 把“自定义的结构”也写成同样的形式

它解决什么问题

(D, R) 只说了“有什么、什么关系”，却没说“能对它做什么”。可是写程序时，我们关心的恰恰是“能做什么”：整数能相加，字符串能拼接，栈能入栈出栈。把“值”和“能做的操作”捆在一起，就是数据类型。

定义 （K3）

数据类型：一组性质相同的**值的集合**，以及定义在这个值集合上的**一组操作**的总称。
抽象数据类型（ADT，教学补全）：一个数学模型以及定义在该模型上的一组操作，通常写成三元组 (D, S, P)——D 是数据对象，S 是 D 上的关系集，P 是对 D 的基本操作集。“抽象”指只规定做什么，不规定怎么做。

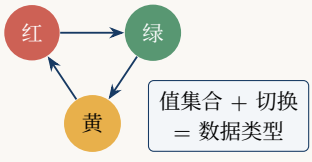
图 7 的大框强调：**只有值、没有操作，不是数据类型**。同一组值配上不同操作就是不同的类型——例如 `int` 有 `%`，`double` 没有。右侧的 ADT 把这一思想推广到自己设计的结构，并把“用什么实现”留到后面决定。

例题 1.4 `int` 的除法

问题：在 C++ 中计算 `7/2` 和 `7%2`。**过程：**`int` 的除法是整数除法，舍去小数部分得 `3`；取余得 `1`。**检查：**$3 \times 2 + 1 = 7$。**说明：**操作的含义由“类型”决定，同样写 `/`，`7.0/2` 得 `3.5`。

生活中的它　红绿灯

场景：路口的交通信号灯。
对应：值集合 {红, 绿, 黄}；操作“切换到下一状态”（我国常见顺序：红 → 绿 → 黄 → 红）。两者合起来就是一个“数据类型”。
小判断：从“红”开始切换 4 次：红 → 绿 → 黄 → 红 → 绿，结果是“绿”。
边界：真实信号灯还有倒计时、闪烁、左转箭头等状态，值集合远不止 3 个。



易错点：三个“数据 xx”分不清

概念	本质
数据项	一个格子：最小单位
数据结构	元素 + 关系
数据类型	值集合 + 操作

笔记原句“数据的不可分割的单位是数据类型”就是把第 1 行和第 3 行混在了一起。区分的关键词：**单位 / 关系 / 操作**。

对照：C++ 的 `int` ↔ 数学中的整数集 $\mathbb{Z}$（同名异义型）

一句话定位：二者都叫“整数”、都有加减乘除，但 `int` 是有限的，$\mathbb{Z}$ 是无限的，很多数学结论搬过来会出错。

维度	$\mathbb{Z}$	`int`（32 位常见）
值集合	无限	$[-2^{31}, 2^{31}-1]$
$a+b$	总有定义	可能溢出
$7 \div 2$	3.5（结果不在 $\mathbb{Z}$）	3



$\mathbb{Z}$ 中的结论	迁移	理由或反例
加法交换律 $a+b=b+a$	△	不溢出时成立；有符号溢出在 C++ 中是未定义行为
$a+b$ 总在集合内（封闭性）	✗	`2147483647+1` 超出 `int` 范围
$(a \div b) \times b = a$	✗	`(7/2)*2` 等于 6

学完这里，你应该能够回答

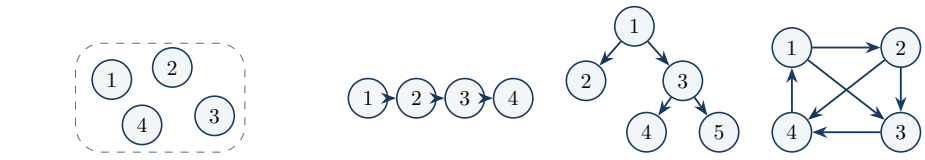
①数据类型由哪两部分组成? ②为什么说“只有值没有操作不是数据类型”? ③ ADT 的“抽象”抽掉了什么? ④ `7/2` 在 C++ 中为什么得 3?

1.4　K4 逻辑结构：集合、线性、树形、图形

学科坐标：第 1 章；前置：K2 的 (D, R)；后续：第 2–3 章线性表、栈、队列（线性），第 5 章树，第 6 章图；相关：离散数学、图论。

它解决什么问题

(D, R) 里的 R 千变万化。如果每遇到一种都单独研究，就没完没了。好在实际问题里的关系，按“一个元素最多有几个前驱、几个后继”来分，只有四种典型形状。分清这四类，后面整门课的章节划分就一目了然。



集合　　线性结构　　树形结构　　图形结构

除“同属一个集合”外无关系　　一对一　　一对多　　多对多

图 8：四类逻辑结构：只看箭头，不看元素内容

图 8 中，从左到右箭头越来越“自由”：集合没有箭头；线性结构中每个点最多出一条、进一条箭头；树形结构中每个点最多进一条箭头，但可以出多条；图形结构进出都不限，还可以绕回来形成环。

定义 （K4）四类逻辑结构

集合：元素之间除“同属于一个集合”外，别无其他关系。
线性结构：元素之间是**一对一**关系。非空时，有且仅有一个“第一个”元素（无前驱）和一个“最后一个”元素（无后继）；其余每个元素**恰有一个直接前驱和一个直接后继**。
树形结构：元素之间是**一对多**关系。有且仅有一个根（无前驱）；其余每个元素**恰有一个直接前驱**，可有零个或多个直接后继。
图形结构（网状结构）：元素之间是**多对多**关系，任意两个元素之间都可能有关系。

补充分类（教学补全）：线性结构称为线性，树形与图形结构称为非线性结构。

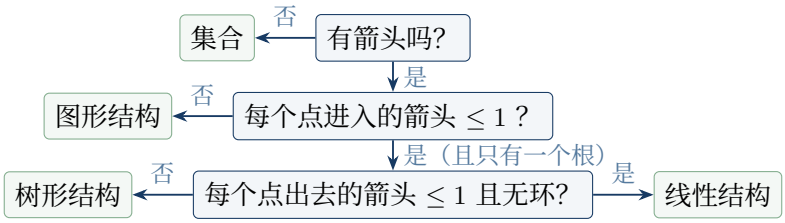


图 9：判定树：拿到一张“点 + 箭头”图，按这三个问题依次判断（只适用于连通且有唯一起点的典型情形）

正例与反例：环形不是线性结构

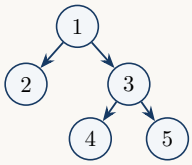


图 10：同样“每个点一进一出”，右图多了箭头 $4 \to 1$ 形成环，于是没有无前驱的元素，不满足线性结构定义

这个反例说明定义中“有且仅有一个第一个元素”这一条件**不能删**：若删去它，环形也会被误判成线性结构。

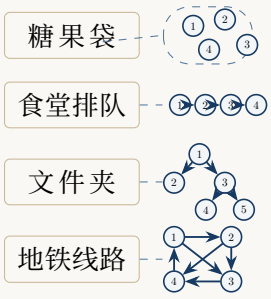
例题 1.5 判断结构类型

已知：$D = \{1,2,3,4,5\}$，$R = \{\langle 1,2\rangle, \langle 1,3\rangle, \langle 3,4\rangle, \langle 3,5\rangle\}$。
第一步：画点连箭头（右图）。**第二步：**统计“进入”的箭头：1 为 0，其余都为 1 ⇒ 唯一根，其余唯一前驱。
第三步：1 和 3 各有 2 个后继 ⇒ 一对多 ⇒ **树形结构**。
本题想说明：判断类型只看“前驱个数”和“后继个数”。



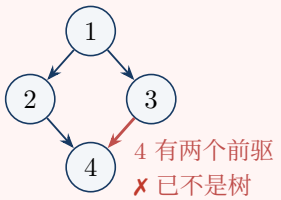
生活中的它　身边的四类结构

场景与对应：一袋混装糖果（集合：彼此没有顺序）；食堂排队（线性：前一个、后一个都唯一）；电脑的文件夹（树形：每个文件夹只有一个上级，可以有很多下级）；地铁线路图（图形：一个换乘站连向多个方向）。
小判断：为什么电脑里每个文件的“路径”是唯一的？因为文件夹是树形结构，从根往下每一步只有唯一的上级，倒推回根的路只有一条。
边界：快捷方式、硬链接可以让同一文件出现在两个位置，此时文件系统严格说已不是纯树形。



易错点

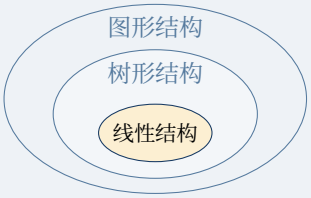
错误想法：“树形结构是一对多，所以一个结点可以有多个前驱。”
错在哪里：“一对多”的“一”指前驱只有一个，“多”指后继可以多个。若一个结点有两个前驱（右图红色箭头），它就变成了图形结构。
正确想法：树中**除根以外，每个结点恰有一个前驱**。



对照：逻辑结构 ↔ 图论中的“图、树、路径”（推广型）

一句话定位：图形结构就是图论中的有向图；树形结构是一种特殊的图（有唯一根、无环）；线性结构又是一种特殊的树（每个结点至多一个后继）。

图论 / 一般图中的做法	迁移	理由或反例
线性结构的结论用于树（如“从头走到尾”）	✗	树有分叉，从根出发不止一条路
树的结论用于线性结构	✓	线性结构是“每层只有一个结点”的树
树的遍历方法用于一般图	△	图可能有环，必须标记“已访问”，否则会绕圈不停



不同及根源：“包含关系”只就“形状”而言；集合结构没有箭头，不在这个嵌套链中。

学完这里，你应该能够回答

①四类逻辑结构各是几对几？②线性结构的定义里，哪个条件把“环”排除在外？③树中除根外每个结点有几个前驱？④文件夹为什么是树形？

1.5　K5 存储结构：顺序、链式、索引、散列

学科坐标：第 1 章；前置：C++ 数组、指针、内存地址；后续：顺序表与链表（第 2 章）、索引查找与散列表（第 7 章）。

它解决什么问题

逻辑结构是画在纸上的“点和箭头”，可内存只是一排编了号的格子，**没有箭头**。存储结构要回答：箭头（关系）在内存里用什么来表示？办法只有两类：要么靠“位置挨着”，要么靠“额外记下地址”。

定义 （K5）四种存储结构

顺序存储：把逻辑上相邻的元素存放在**物理位置上也相邻**的存储单元中，元素间的关系由存储单元的相邻关系体现（通常借助数组实现）。
链式存储：**不要求**逻辑上相邻的元素在物理位置上也相邻，元素间的关系借助**指示元素存储地址的指针**来表示。
索引存储：存储元素的同时，另建一张**索引表**，索引项一般形如“(关键字，地址)”。
散列存储：根据元素的**关键字直接计算**出该元素的存储地址（又称哈希存储）。
其中顺序存储和链式存储是最基本、最常用的两种（笔记红笔所记）。

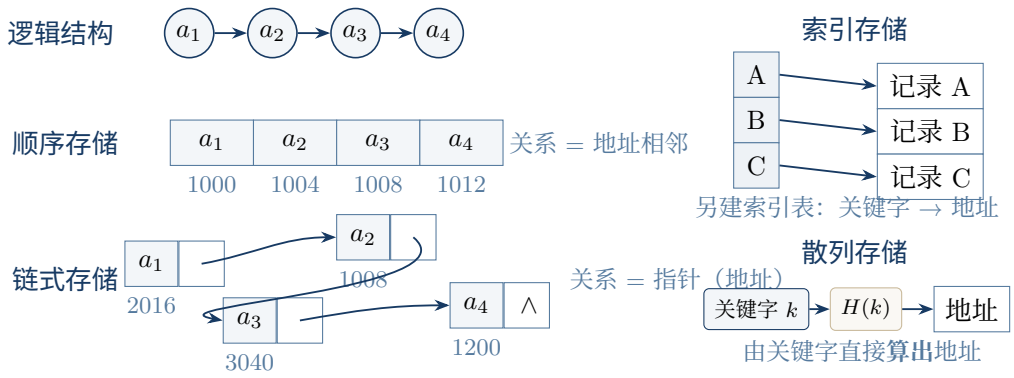


图 11: 同一个线性结构 $a_1 \to a_2 \to a_3 \to a_4$ 的不同存法（地址为示意值）。左上为逻辑图；顺序存储靠“挨着”；链式存储每个结点多一格存“下一个在哪”，∧ 表示空指针；右侧为另外两种方法

顺序存储的地址公式：为什么能“一步到位”

核心公式 （顺序存储定位）

设第 1 个元素的起始地址为 $\mathrm{LOC}(a_1)$，每个元素占 L 个存储单元，则

$$\mathrm{LOC}(a_i) = \mathrm{LOC}(a_1) + (i-1) \times L, \qquad 1 \le i \le n$$

符号： i 为逻辑位序（从 1 开始）；L 单位为字节；LOC 为起始地址。**条件：** 元素大小相同且连续存放。

推导

要证： 上式对所有 $1 \le i \le n$ 成立。**依据：** 顺序存储的定义——a_{i+1} 紧挨在 a_i 之后，因此 $\mathrm{LOC}(a_{i+1}) = \mathrm{LOC}(a_i) + L$。

过程（数学归纳法）： $i = 1$ 时右边 $= \mathrm{LOC}(a_1) + 0$，成立。假设对 i 成立，则

$$\begin{aligned} \mathrm{LOC}(a_{i+1}) &= \mathrm{LOC}(a_i) + L && \text{（相邻存放）} \\ &= \mathrm{LOC}(a_1) + (i-1)L + L && \text{（归纳假设）} \\ &= \mathrm{LOC}(a_1) + \big((i+1)-1\big)L. && \text{（整理）} \end{aligned}$$

故对 $i+1$ 也成立。证毕。

直观： 要到第 i 个，前面有 $i-1$ 个元素挡着，每个宽 L。



图 12: 地址公式的几何意义：第 i 个元素的地址 = 起点 + 跳过的宽度

例题 1.6 求地址

已知： $\mathrm{LOC}(a_1) = 100$，$L = 8$。**求：** $\mathrm{LOC}(a_6)$。

代入： $100 + (6-1) \times 8 = 140$。**检查：** a_2 在 108，每次加 8：108、116、124、132、140，第 5 次加完正是 a_6。

说明： 无论 i 多大，都只算一次乘法和加法——顺序存储可以**随机存取**。

顺序与链式：相近概念辨析

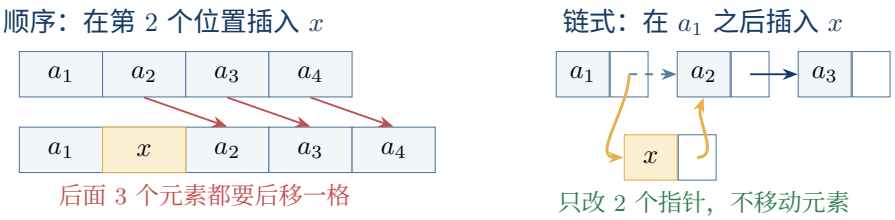
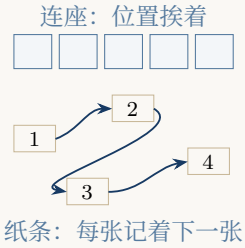


图 13: 同一个“插入”操作：顺序存储要搬家，链式存储只需改指针（虚线为被替换的旧指针）

比较维度	顺序存储	链式存储
关系怎么表示	物理位置相邻	指针
按位置访问第 i 个	地址公式一步算出，$O(1)$	从头沿指针走 $i-1$ 步，$O(n)$
在中间插入 / 删除	平均要移动约一半元素，$O(n)$	找到位置后只改指针，$O(1)$
空间	只存数据，密度高；需预先分配	每个结点多存一个指针；按需申请
典型实现	数组	结点 + 指针

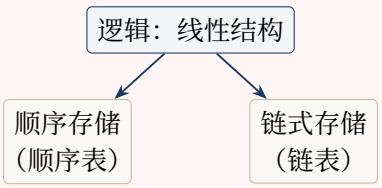
生活中的它　电影院连座 vs 寻宝游戏

场景： 8 个同学看电影买了连座；另一种玩法是寻宝游戏，每张纸条上写着下一张纸条藏在哪里。
对应： 连座 ↔ 顺序存储（座位号连续 = 地址相邻）；纸条 ↔ 链式存储（纸条上的“下一个地点” = 指针）。
小估算： 第 1 人坐 5 排 3 座，第 8 人直接算出 5 排 $3+7=10$ 座，一步；寻宝要找第 8 张纸条，必须先依次找到前 7 张。
边界： 真实影院座位有过道，号码不一定连续，就像实际内存还有对齐和碎片。



易错点

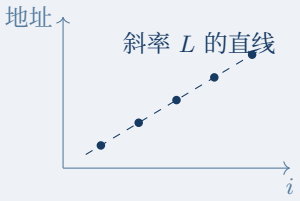
错误想法 1：“链式结构是第五种逻辑结构。” **正确：** 顺序、链式、索引、散列都是**存储**结构；逻辑结构只有集合、线性、树形、图形四类。
错误想法 2：“线性结构只能顺序存储。” **正确：** **同一种逻辑结构可以采用不同的存储结构**（右图）。逻辑结构是“想法”，存储结构是“落实”。



对照：顺序 / 链式存储 ↔ 数列的通项公式 / 递推公式（类比同构型）

一句话定位： 顺序存储的地址公式就是一个**等差数列通项公式**；链式存储“只知道下一个在哪”则像**递推公式**。

维度	数列（高中 / 高数）	存储结构
直接定位	通项 $a_i = a_1 + (i-1)d$	$\mathrm{LOC}(a_i) = \mathrm{LOC}(a_1) + (i-1)L$
逐项定位	递推 $a_{i+1} = f(a_i)$	指针 `p=p->next`
求第 i 项代价	通项 1 步；递推 $i-1$ 步	顺序 $O(1)$；链式 $O(n)$



数列中的结论	迁移	理由或反例
有通项就能直接求任意一项	✓	顺序存储可随机存取
首项下标是 1	△	C++ 数组下标从 0 开始，此时公式变为 $\mathrm{LOC}(a[i]) = \mathrm{LOC}(a[0]) + i \cdot L$
递推数列也能“直接写出第 i 项”	✗	一般链表的地址毫无规律，只能顺着指针走

学完这里，你应该能够回答

①四种存储结构各用什么表示关系？②写出顺序存储地址公式并说明为什么是 $i-1$。③按位置访问、中间插入，顺序与链式各快在哪里？④“链式结构”属于逻辑结构吗？

本章知识地图

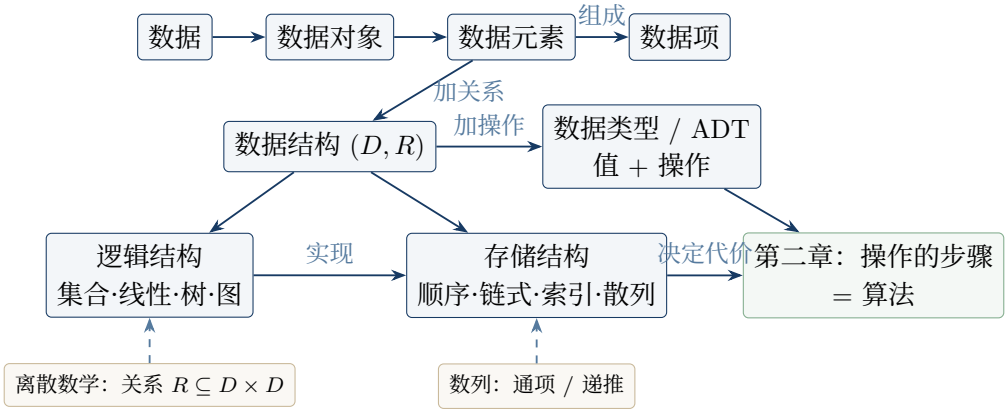


图 14: 第一章知识地图（虚线：与其他课程知识的对应）

到这里，我们已经解决了什么？下一步还需要解决什么？

我们已经能精确描述“有哪些数据、它们之间什么关系、在内存里怎么放”。但数据放好之后总要用：查找、插入、排序……每一种“用法”都是一串步骤。什么样的步骤才是合格的？同一件事有两种步骤时，怎样判断谁更好？这就是第二章“算法”要回答的问题。

第二章　算法与算法分析

本章回答三个问题：①什么样的“步骤”才配叫算法？②两个都正确的算法，怎样比较好坏？③不运行程序，怎样预估它“随 n 变大会慢多少”——笔记中的 $O(\log_2 n)$ 是怎么来的？

本章知识路线：算法定义 → 五个特性 → 好算法的标准 → 语句频度 → 大 O 记号 → 常见阶 → 笔记例题完整分析 → 空间复杂度。

2.1　K6 算法的定义与五个特性（Algorithm）

学科坐标：第 1 章；前置：C++ 的顺序、分支、循环；后续：此后每一章的每一个操作（插入、查找、排序）都是一个算法。

它解决什么问题

上一章把数据“放好”了，可“怎么用”还没说。生活里“做一件事的步骤”多种多样，有的写得含糊（“盐少许”），有的永远做不完（“一直等下去”）。计算机只能执行明确的、做得完的步骤，所以需要给“合格的步骤”划一条线。

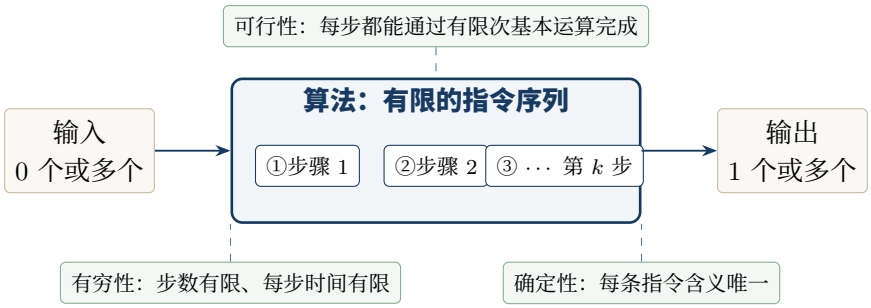


图 15: 算法的五个特性在“输入 → 步骤 → 输出”图上的位置：两端是输入、输出，中间三个约束步骤本身

定义 （K6）

算法：对特定问题求解步骤的一种描述，它是指令的有限序列，其中每一条指令表示一个或多个操作。算法具有五个重要特性：
①**有穷性：**算法必须在执行**有穷步**之后结束，且**每一步**都可在有穷时间内完成。
②**确定性：**每条指令都有确切含义，**无二义性**；相同输入只能得到相同输出。
③**可行性：**算法中的操作都可以通过已经实现的基本运算执行**有限次**来实现。
④**输入：**有**零个或多个**输入。　　⑤**输出：**有**一个或多个**输出。

为什么输入可以是 0 个，输出却至少 1 个？输入为 0：例如“输出 100 以内的全部素数”，问题本身已经说清了，不需要外部数据。输出至少 1：没有输出的“算法”对外界没有任何结果，等于什么也没解决。

反例：逐条去掉一个特性会怎样

违反有穷性
`x=1;`
`while(x>0) x=x+1;`
（数学上 x 永远大于 0）

违反确定性
“把成绩较好的同学排在前面”
（多好算“较好”？）

违反可行性
“写出 π 的全部小数位，再取第 10 位”
（全部位数无法写完）

违反输出
“计算 $1+\cdots+n$”，算完不打印、不返回
（对外毫无结果）

图 16: 四个“像算法但不是算法”的描述。注意第一个：在真实计算机上 `int` 会溢出，但我们按描述的意图判断——它设计上就不打算停

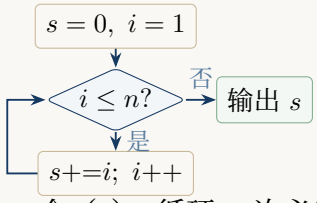
证明：笔记中的程序满足有穷性

证明（教学补全）

要证：对任意正整数 n，程序 `t=1; while(t<=n) t=t*2;` 一定会结束。
思路：t 每次翻倍，翻倍的增长没有上限，迟早会超过任何固定的 n。
详细：设循环体已执行 k 次，则 $t=2^k$（初值 $2^0=1$，每次乘 2，由归纳可得）。由于 $2^k>k$ 对所有 $k\geq 0$ 成立，取 $k=n$ 时 $t=2^n>n$，循环条件 `t<=n` 不再成立。因此循环体至多执行 n 次，程序必定结束；每一次执行只做一次比较与一次乘法，时间有限。证毕。
一句话：“每轮都向终点靠近，且一定能到达”就是证明有穷性的通用思路。（实际执行次数远少于 n，见第 2.4 节。）

例题 2.1 同一个算法的三种描述

问题：求 $1+2+\cdots+n$。
自然语言：令 $s=0$；让 i 从 1 取到 n，每次把 i 加到 s 上；输出 s。




```
int sum(int n){
  int s = 0;
  for(int i=1;i<=n;i++) s+=i;
  return s;
}
```

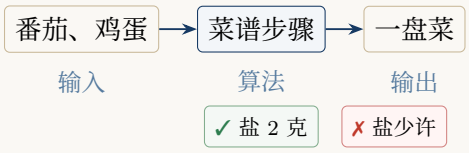
检查五特性：输入 1 个（n），输出 1 个（s），循环 n 次必结束，每步含义明确且可执行。**说明：**算法与描述语言无关，流程图、伪代码、C++ 都只是“写法”。

相近概念：算法 vs 程序

维度	算法	程序
本质	解题步骤（思想）	用某种语言对算法的实现
描述方式	自然语言、流程图、伪代码、程序设计语言均可	只能用程序设计语言
有穷性	必须满足	不一定：操作系统、服务器程序设计上就要一直运行

生活中的它　菜谱

场景：照着菜谱做番茄炒蛋。
对应：食材 ↔ 输入；一盘菜 ↔ 输出；步骤有限 ↔ 有穷性；“盐 2 克” ↔ 确定性；“用锅翻炒” ↔ 可行性（厨房里做得到）。
小判断：“盐少许”违反了哪条？**确定性**——两个人对“少许”的理解不同，同样的输入会得到不同的结果。
边界：人做菜可以凭经验补全含糊之处，计算机不能，所以算法的要求比菜谱严格得多。



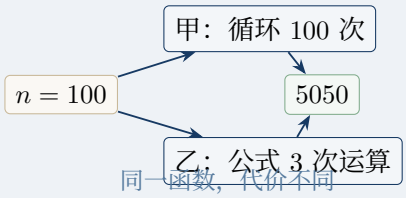
易错点

错误想法：“程序能编译运行，就是一个算法。”　**错在哪里：**程序可能永不停止（死循环），不满足有穷性。　**正确想法：**算法 $\neq$ 程序；判断是否是算法，要**逐条核对五个特性**，其中最常被违反的是**有穷性和确定性**。

对照：算法 ↔ 数学中的“函数”（同名异义型）

一句话定位：函数只规定“输入对应哪个输出”，算法还规定“怎样一步步算出来”。同一个函数可以有很多个算法。

同一个小算例：函数 $f(n) = 1 + 2 + \cdots + n$。
算法甲：循环累加，做 n 次加法。
算法乙：用公式 $n(n+1)/2$，只做 1 次乘法、1 次加法、1 次除法。
$n = 100$ 时两者都输出 5050，但甲做了 100 次加法，乙只做 3 次运算。



关于函数的结论	迁移	理由或反例
两个函数对每个输入输出都相同，则它们相等	✓	两个算法输出处处相同，称它们“计算同一个函数”，都正确
函数相等，则“一样好”	✗	算法甲、乙计算同一函数，但运算次数差别巨大
函数可以在无穷集合上“直接给出”	✗	算法必须在有穷步内给出每个输出（有穷性）

学完这里，你应该能够回答

①五个特性是什么？输入、输出各至少几个？②有穷性包含哪两层意思？③为什么操作系统是程序却不是算法？④“盐少许”违反哪个特性？

2.2　K7 好算法的标准

学科坐标：第 1 章；前置：K6；后续：K8、K9 把“高效”量化；相关：软件工程、控制工程中的“鲁棒性”。

它解决什么问题

满足五个特性只说明“它是算法”，不说明“它好”。同一个问题往往有多个算法，需要一套标准来选。笔记中写：以算法执行所耗费的时间与占用的空间来判断——这是“效率”层面；但在效率之前，还有更基本的要求。



图 17：好算法的四根“柱子”。笔记中的“高效性”“低存储性”合为最右一根

定义 （K7）好算法的四条标准

正确性：对一切合法输入，都能得到满足要求的结果。
可读性：便于人阅读、理解和交流（笔记写作“易读性”）。
健壮性（鲁棒性）：输入非法数据时，能适当地作出反应（如报错），而不是产生莫名其妙的结果或崩溃。
高效率与低存储量需求：执行时间短（时间复杂度低），所需存储空间少（空间复杂度低）。

例题 2.2 正确但不健壮的程序

```
// 原版：n=0 时除以 0，程序出错
double avg(int a[], int n){
  int s=0;
  for(int i=0;i<n;i++) s+=a[i];
  return (double)s/n;
}
```

```
// 改进：先检查非法输入
double avg(int a[], int n){
  if(n<=0){ cerr<<"n␣必须为正"; return 0; }
  int s=0;
  for(int i=0;i<n;i++) s+=a[i];
  return (double)s/n;
}
```

分析：对合法输入（$n \geq 1$）两版结果相同，都“正确”；但左版对 $n = 0$ 会做除以 0 的运算，右版先判断并报错——**健壮性**的差别只在非法输入上显现。

生活中的它　导航软件

场景：用导航软件去学校图书馆。
对应：能把你带到图书馆 ↔ 正确性；输入一个不存在的地址时提示“未找到”而不是闪退 ↔ 健壮性；几秒内规划出路线 ↔ 高效率；不占满手机内存 ↔ 低存储。
小判断：一个“路线最优但要算 10 分钟”的导航与“路线稍差但 2 秒出结果”的导航，日常出行多数人选后者——高效率往往与“最优”需要权衡。
边界：商业导航还要考虑实时路况等，远不止这四条标准。



易错点

错误想法：“健壮性就是正确性。”　**区别：**正确性只管**合法输入**的结果对不对；健壮性管**非法输入**时会不会出乱子。例 2.2 的左版“正确但不健壮”。另一个常见误区是“越快越好”：如果为了快牺牲了正确性，再快也没有意义。

对照：算法的“健壮性” ↔ 控制工程中的“鲁棒性”（同名异义型）

一句话定位：二者英文都是 robustness，核心思想都是“偏离理想条件时仍能正常工作”，但偏离的对象与衡量方式不同。（控制部分为**拓展联想：以下内容只用于建立知识联系，当前阶段不要求掌握**。）

维度	算法健壮性	控制系统鲁棒性
偏离什么	输入数据不合法（空表、负数、格式错）	被控对象参数与模型不一致、存在扰动
期望表现	给出合理反应（报错、拒绝），不崩溃	系统仍保持稳定并满足性能指标
怎样衡量	通常靠测试与边界检查，无统一数值	可用稳定裕度等指标定量描述

迁移的结论	迁移	理由
“考虑最坏情况而不只考虑理想情况”	✓	两边的共同思想
“健壮性可以用一个数值打分”	✗	本课程中健壮性主要是定性要求
“只要正常工况下表现好，就算鲁棒/健壮”	✗	两边都强调非理想条件

学完这里，你应该能够回答

①好算法的四条标准？②“正确”和“健壮”分别针对什么输入？③笔记里说“以时间与空间判断优劣”，对应哪条标准？

2.3　K8 时间复杂度（Time Complexity）

学科坐标：第 1 章，本讲最重要的知识点；前置：对数、求和、极限；后续：此后每一个算法都要写出时间复杂度；相关：高等数学“无穷大的阶”。

它解决什么问题

想比较两个算法谁快，最直接的办法是“跑一下、掐秒表”。可这样测出的时间依赖于电脑快慢、编译器、同时开着的其他软件，换一台电脑结论可能就变了。我们需要一个**不依赖机器**的衡量方法。思路是：机器再快，每条基本语句也要执行一次；**数一数基本语句执行了多少次**，再看这个次数随问题规模 n 怎样增长。

第一步：语句频度与 $T(n)$

定义　（K8-1）问题规模、语句频度、$T(n)$

问题规模 n：输入量的大小，如表长、矩阵阶数。
语句频度：一条语句在算法中被重复执行的次数。
基本语句：算法中执行次数最多、对总时间起决定作用的语句（通常是最内层循环体）。
$T(n)$：算法中基本运算的执行次数，表示为 n 的函数。笔记的说法“一般将算法基本运算执行次数作为时间复杂度衡量标准”说的就是它。

例题 2.3 数一数

```
1  for(i=1;i<=n;i++) // 频度 n+1（多判断一次）
2    for(j=1;j<=3;j++) // 频度 n*(3+1)
3      x++; // 频度 3n 基本语句
```



结果：$T(n)=3n$。**说明：**内层循环次数是常数 3，不随 n 变化，所以整体只是 n 的 3 倍。下面会看到，“3 倍”在复杂度里会被忽略。

第二步：只看增长趋势——大 O 记号

$T(n)=3n^2+5n+100$ 这样的式子太细了。当 n 很大时，$3n^2$ 远远压过 $5n+100$，而系数 3 只反映“机器每步多快”。我们真正关心的是**增长的量级**，于是引入大 O。

核心公式　（K8-2）大 O 记号的定义

$$T(n)=O\big(f(n)\big)\iff \exists c>0,\ \exists n_0,\ \text{使得对一切 } n\ge n_0,\ \ 0\le T(n)\le c\cdot f(n)$$

读作“$T(n)$ 的阶不超过 $f(n)$”。算法的**（渐近）时间复杂度**记作 $T(n)=O(f(n))$，通常取能满足定义的、尽量“紧”的简单函数 $f(n)$。

图 18 中，横轴是规模 n，纵轴是执行次数。实线是精确的 $T(n)$，粗虚线是 $4n^2$。在 n_0 左边，$T(n)$ 反而更大（因为有个 $+100$）；但从阴影区开始，实线永远在虚线下面——这正是定义中“存在 c 和 n_0”的含义。

证明：$3n^2+5n+100=O(n^2)$

要证：存在 c、n_0，使 $n\ge n_0$ 时 $3n^2+5n+100\le cn^2$。
思路：把低次项都“放大”成 n^2 的倍数。
方法一（取 $c=108$，$n_0=1$）：当 $n\ge 1$ 时，$5n\le 5n^2$（因为 $n\le n^2$），$100\le 100n^2$（因为 $n^2\ge 1$），

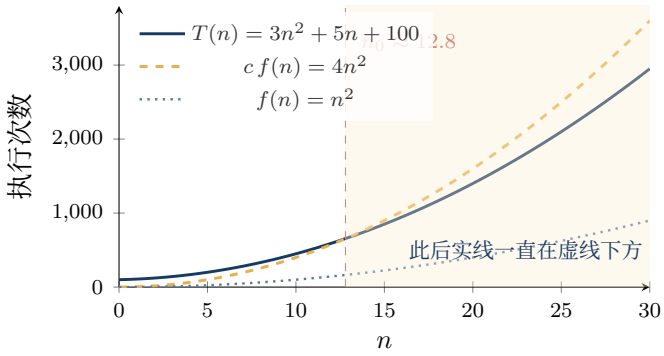


图 18: 大 O 的几何意义：从某个 n_0 起，$T(n)$ 的曲线始终被 $c \cdot f(n)$ 的曲线压在下面。n_0 之前谁大谁小无所谓

所以

$$3n^2 + 5n + 100 \le 3n^2 + 5n^2 + 100n^2 = 108\,n^2.$$

方法二（取 $c = 4$）： $3n^2 + 5n + 100 \le 4n^2 \iff n^2 - 5n - 100 \ge 0 \iff n \ge \dfrac{5 + \sqrt{425}}{2} \approx 12.8$，取 $n_0 = 13$ 即可（图 18）。

结论： $T(n) = O(n^2)$。两种方法说明 c 与 n_0**不唯一**，只要存在一组即可。

两条常用法则（含证明）

法则 1（多项式取最高次）： 若 $T(n) = a_k n^k + \cdots + a_1 n + a_0$，$a_k > 0$，则 $T(n) = O(n^k)$。

证：$n \ge 1$ 时，$|a_i n^i| \le |a_i| n^k$，于是 $T(n) \le (|a_k| + \cdots + |a_0|)\, n^k$，取 $c = \sum |a_i|$，$n_0 = 1$。

法则 2（加法与乘法规则）： 若 $T_1 = O(f)$，$T_2 = O(g)$，则

顺序执行：$T_1 + T_2 = O(\max\{f, g\})$；　嵌套执行：$T_1 \cdot T_2 = O(f \cdot g)$。

证（加法）：$n \ge \max(n_1, n_2)$ 时，$T_1 + T_2 \le c_1 f + c_2 g \le (c_1 + c_2)\max\{f, g\}$。（乘法）：同理 $T_1 T_2 \le c_1 c_2\, fg$。

第三步：常见的阶

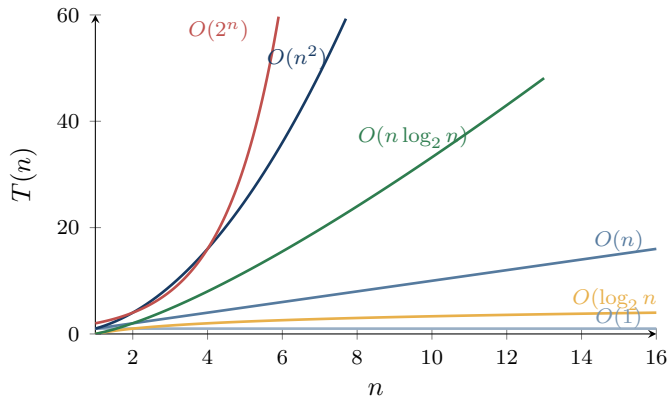


图 19： 常见阶的增长曲线（同一坐标系）。n 稍一增大，曲线之间的差距就迅速拉开

$$O(1) < O(\log_2 n) < O(n) < O(n\log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

n	$\log_2 n$	n	$n\log_2 n$	n^2	n^3	2^n
10	3.3	10	33	100	1000	1024
100	6.6	100	664	10^4	10^6	$\approx 1.3\times 10^{30}$
1000	10.0	1000	9966	10^6	10^9	天文数字

表中最值得看的是 $\log_2 n$ 这一列：n 从 10 增长到 1000（扩大 100 倍），$\log_2 n$ 只从 3.3 增长到 10。这就是笔记例题中 $O(\log_2 n)$ “非常快”的含义。

> **教学补全 ： 对数的底数无关紧要**
>
> 由换底公式 $\log_a n = \dfrac{\log_b n}{\log_b a}$，不同底数的对数只差一个常数倍 $\dfrac{1}{\log_b a}$，而大 O 忽略常数倍，所以 $O(\log_2 n) = O(\log_{10} n) = O(\ln n)$，常简写为 $O(\log n)$。笔记写的 $O(\log_2 n)$ 是因为它直接来自“乘 2”的过程。

第四步：笔记例题的完整分析

```c
int algorithm(int n){
    int t = 1;
    while (t <= n) // 循环条件
        t = t * 2; // 基本语句
    return t;
}
```

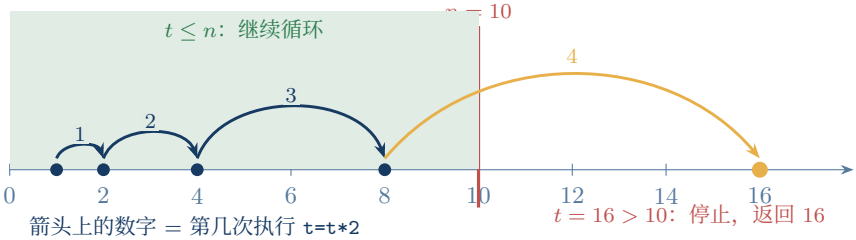


图 20： $n = 10$ 时 t 的变化：$1 \to 2 \to 4 \to 8 \to 16$。每个箭头是一次 `t=t*2`，共 4 次；第 5 次判断 $16 \le 10$ 不成立，退出

第几次判断	1	2	3	4	5
判断时 t	1	2	4	8	16
$t \le 10$?	是	是	是	是	否
执行 `t*=2` 后	2	4	8	16	（退出）

精确次数与复杂度（教学补全：把笔记的“$2^x = n$”做严谨）

要求： $n \ge 1$ 时循环体执行次数 $T(n)$，并证明 $T(n) = O(\log_2 n)$。
第 1 步（t 的通项）： 循环体执行 k 次后 $t = 2^k$。依据：初值 $2^0 = 1$，每次乘 2。
第 2 步（何时继续）： 第 $k+1$ 次判断时 $t = 2^k$。循环体会执行第 $k+1$ 次 $\iff 2^k \le n \iff k \le \log_2 n \iff k \le \lfloor \log_2 n \rfloor$（$k$ 是整数）。
第 3 步（计数）： 满足条件的 k 为 $0, 1, \ldots, \lfloor \log_2 n \rfloor$，共

$$\boxed{T(n) = \lfloor \log_2 n \rfloor + 1}$$

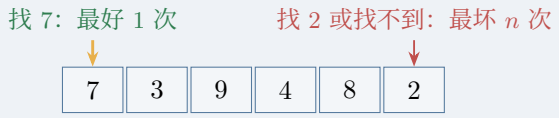
个。验算：$n = 10$，$\lfloor \log_2 10 \rfloor + 1 = 3 + 1 = 4$，与图 20 一致。
第 4 步（化为 O）： 当 $n \ge 2$ 时 $\log_2 n \ge 1$，所以 $T(n) \le \log_2 n + 1 \le 2 \log_2 n$，取 $c = 2$，$n_0 = 2$，得 $T(n) = O(\log_2 n)$。
附：返回值为 $2^{\lfloor \log_2 n \rfloor + 1}$，即**大于 n 的最小的 2 的幂**（$n = 10$ 返回 16）。
一句话： 笔记中 $2^x = n \Rightarrow x = \log_2 n$ 是“抓量级”的快捷写法；精确次数要加 1 并取整，但这不改变阶。

方法总结：求时间复杂度的四步

①找基本语句（通常是最内层循环体） → **②算它的频度**（列出循环变量的变化规律，必要时写出 $T(n)$） → **③只留最高阶项、去掉系数** → **④写成** $O(\cdot)$。
常见模式：循环变量 `i++` 走到 n $\Rightarrow$ n 次；`i*=2` 或 `i/=2` $\Rightarrow$ 约 $\log_2 n$ 次；两层独立循环 $\Rightarrow$ 相乘；先后两段 $\Rightarrow$ 取大者。

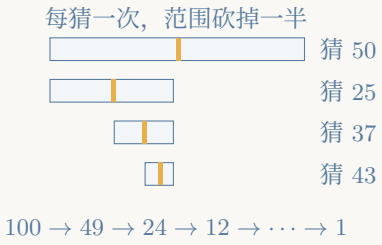
教学补全 ： 最好、最坏与平均情况

有的算法执行次数不仅取决于 n，还取决于输入的具体内容。例如在 n 个元素中**顺序查找**某个值：第一个就是它时只比较 1 次（最好情况）；在最后或不存在时比较 n 次（最坏情况）；若每个位置等可能，平均比较 $\frac{1+2+\cdots+n}{n} = \frac{n+1}{2}$ 次（平均情况）。**一般讨论最坏情况**，因为它给出了运行时间的上限。顺序查找的时间复杂度为 $O(n)$。



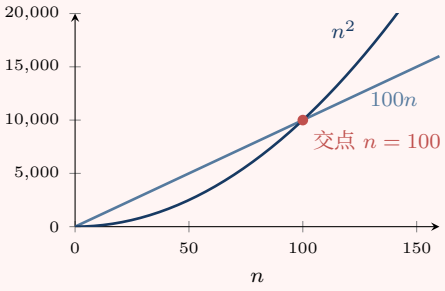
生活中的它　猜数字游戏

场景：朋友心里想一个 1 到 100 之间的整数，你每猜一次，他只说“大了”或“小了”。
对应：可能范围的大小 $\leftrightarrow$ 规模 n；每次猜中间值、范围减半 $\leftrightarrow$ 笔记例题中 t 每次翻倍的“倒过来”；猜的次数 $\leftrightarrow$ $T(n)$，约 $\log_2 n$。
小估算：$2^6 = 64 < 100 \le 128 = 2^7$，所以每次都猜中点时，最多 7 次必中；范围扩大到 1 到 1000，也只需最多 10 次（$2^{10} = 1024$）。
边界：前提是对方如实回答，且你每次都能精确猜中点；人凭感觉猜往往做不到。



易错点（三个高频错误）

①**写成** $O(2n)$、$O(n^2+n)$。大 O 中应去掉系数和低阶项，规范写法是 $O(n)$、$O(n^2)$。
②**“$O(n^2)$ 的算法在任何时候都比 $O(n)$ 慢”**。错！大 O 描述的是 n 足够大时的趋势。右图中 $100n$ 与 n^2 在 $n = 100$ 处相交，$n < 100$ 时 n^2 反而更小。
③**把 $\log_2 n$ 当作精确次数**。$n = 10$ 时次数是 4，不是 3.32。精确次数与阶要分清。



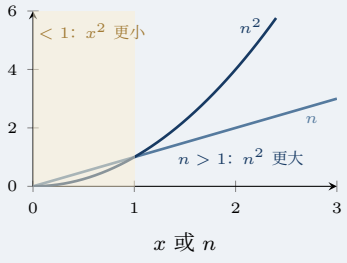
对照：算法中的大 O $\leftrightarrow$ 高等数学中的“无穷大 / 无穷小的阶”（思想重现型）

一句话定位：大 O 记号本来就来自数学分析；高数里用它比较“谁趋于无穷（或零）更快”，算法里用它比较“谁的执行次数增长更快”。二者同源，但**关心的极限方向不同**。

维度	高等数学	本课程
自变量的变化	$x \to 0$（泰勒展开余项）或 $x \to \infty$	只关心 $n \to \infty$
常用记号	o（高阶无穷小）、O、$\sim$（等价）	主要用 O（上界）
“高次项”的地位	$x \to 0$ 时高次项更小，可忽略	$n \to \infty$ 时高次项更大，要保留
同阶的判断	$\lim \frac{f}{g} = c \neq 0$	同上，此时 $f = O(g)$ 且 $g = O(f)$

相同：“忽略常数倍、只比增长快慢”的思想；用极限判断阶的方法。**不同及根源：**高数中的 O 常放在 $x \to 0$ 的语境（如 $\sin x = x + O(x^3)$），此时幂次越高越“小”；算法里 $n \to \infty$，幂次越高越“大”。忘了极限方向，就会把该留的项删掉。

同一个小算例：$g = n^2 + n$（x 同理）。
$n \to \infty$：$\dfrac{n^2+n}{n^2} \to 1$，主导项是 n^2，故 $g = O(n^2)$。
$x \to 0$：$\dfrac{x^2+x}{x} \to 1$，主导项是 x，故 $x^2 + x \sim x$。
同一个式子，“谁说了算”正好相反。



高数中的结论	迁移	理由或反例
$\lim_{n\to\infty}\frac{f}{g}=c\neq 0\Rightarrow$ 同阶	✓	可直接用来判断两个 $T(n)$ 是否同阶
$\lim\frac{f}{g}=0$ 则 f 比 g 低阶	✓	例如 $\lim\frac{\log_2 n}{n}=0$，所以 $\log_2 n$ 增长比 n 慢
“泰勒展开中可以丢掉高次项”	✗	那是 $x\to 0$ 的情形；$n\to\infty$ 时应丢掉低次项
$f=O(g)$ 意味着 f 与 g 同阶	△	大 O 只给上界：$n=O(n^2)$ 也对，只是不“紧”；需加上 $g=O(f)$ 才同阶

学完这里，你应该能够回答

①为什么不用“秒数”而用“执行次数”衡量？②写出大 O 的定义，并说出 c、n_0 的作用。③证明 $2n+100=O(n)$。④笔记例题中 $n=1000$ 时循环执行几次？⑤为什么 $O(\log_2 n)$ 与 $O(\log_{10} n)$ 相同？⑥什么时候 $O(n^2)$ 的算法反而更快？

2.4　K9 空间复杂度（Space Complexity，教学补全）

学科坐标：第 1 章；笔记“以时间与占用空间来判断”中“空间”的量化；后续：排序算法中常见“原地排序”的说法。

它解决什么问题

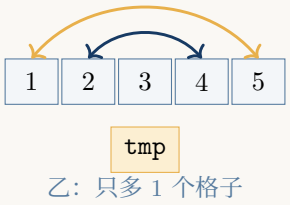
算法运行除了花时间，还要占内存。内存同样是有限资源，所以也要像时间一样，看它随 n 怎样增长。

定义 （K9）

空间复杂度：算法所需**辅助存储空间**（不含输入数据本身所占空间）随问题规模 n 增长的量级，记作 $S(n)=O(f(n))$。若辅助空间是与 n 无关的常数，称算法**原地工作**，$S(n)=O(1)$。

例题 2.4 把数组逆置

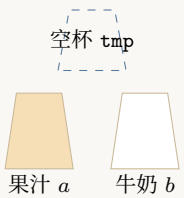
方法甲：新开一个同样大的数组 `b`，倒着抄过去再抄回来——辅助空间 n 个单元，$S(n)=O(n)$。
方法乙：首尾两两交换，只用一个临时变量 `tmp`——$S(n)=O(1)$，原地工作。
两者时间都是 $O(n)$。说明：时间相同的两个算法，空间可以差很多。



乙：只多 1 个格子

生活中的它　交换两杯饮料

场景：一杯果汁、一杯牛奶，想把它们换个杯子装。
对应：两个杯子 ↔ 两个变量；借来的空杯 ↔ 临时变量 `tmp`。
小判断：不管杯子多大，只需借 **1 个**空杯，辅助空间是常数，$O(1)$。若要交换 n 对杯子，依次进行，同一个空杯可反复使用，仍是 $O(1)$。
边界：倒的过程会有残留、杯子大小可能不同，比变量交换复杂。



易错点

错误想法：“数组有 n 个元素，所以任何处理它的算法空间复杂度都是 $O(n)$。”　**正确：**空间复杂度只算**辅助**空间，输入本身占用的空间不计入。例 2.4 方法乙就是 $O(1)$。

相似知识对比：空间复杂度的记号、定义方式与时间复杂度完全相同，与高数“阶”的对照已在 K8 中完成，此处不再重复；需要注意的只是“时间可以重复利用，空间若不释放则会累积”。时间与空间常常可以相互交换（如用额外的表加速查找），这叫**以空间换时间**，第 7 章散列表就是典型例子。

学完这里，你应该能够回答

①空间复杂度计不计入输入数据本身？②什么叫“原地工作”？③举一个时间相同、空间不同的例子。

本章知识地图

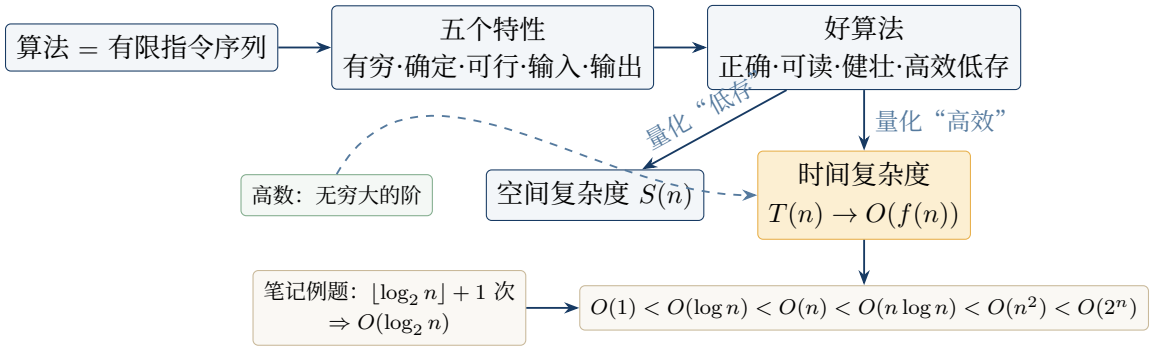


图 21：第二章知识地图（虚线：与高等数学的对应）

到这里，我们已经解决了什么？

两条主线都已打通：第一章给出了“数据怎样组织”，第二章给出了“步骤合格的标准”和“衡量代价的尺子”。以后学习线性表、栈、树、图时，每遇到一个新操作，都可以问三个问题：**它在什么逻辑结构上？用什么存储结构？时间、空间复杂度各是多少？**下一章用 38 道练习检验你是否真的掌握了这些工具。

第三章　正式练习（38 题）

先独立完成，再看第四章的答案与图解。练习分三层：基础 25 题，中等偏下 10 题，跨学科 3 题。题目区的图只画题目已给出的信息，不含答案提示。

知识点—题目对应表

核心知识点	基础题	提升题	跨学科	涉及的对照
K1 数据元素 / 数据项	1, 14, 21	34		C++ 结构体
K2 数据结构 (D, R)	2, 21	34	36	离散数学关系
K3 数据类型 / ADT	16			`int` 与 $\mathbb{Z}$
K4 逻辑结构	2, 3, 11, 17	30	36	图论；控制框图
K5 存储结构	4, 5, 12, 18, 24	27, 30, 32, 34	38	数列通项
K6 算法与五特性	6, 7, 13, 25			数学函数
K7 好算法的标准	20			控制鲁棒性
K8 时间复杂度	8, 10, 15, 19, 22, 23	26, 28, 29, 31, 33, 35	37	高数无穷大的阶；二分法
K9 空间复杂度	9			

3.1　A. 基础巩固训练（25 题）

一、单项选择题（10 题）

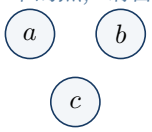
1. 基础　右图成绩表中，数据的**基本单位**是（　）。

学号	姓名	班级	成绩
100872	刘木	1442	92
100873	王一	1442	85
100874	李华	1443	78

A. 数据项　　B. 数据元素　　C. 数据类型　　D. 数据对象

2. 基础　设 $D=\{a,b,c\}$，$R=\{\langle a,b\rangle,\langle a,c\rangle,\langle b,c\rangle\}$，则 (D,R) 属于（　）。

只画出 D 中的点，请自己连箭头



A. 集合　　B. 线性结构　　C. 树形结构　　D. 图形结构

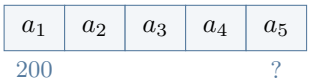
3. 基础　线性结构中，数据元素之间的关系是（　）。

A. 一对一　　B. 一对多　　C. 多对多　　D. 除同属一个集合外无关系

4. 基础　下列选项中，属于**存储结构**的是（　）。

A. 线性结构　　B. 树形结构　　C. 链式结构　　D. 图形结构

5. 基础　线性表采用顺序存储，$\mathrm{LOC}(a_1)=200$，每个元素占 4 个存储单元，则 $\mathrm{LOC}(a_5)=$（　）。



A. 216　　B. 220　　C. 204　　D. 212

6. 基础　下列选项中，**不属于**算法五个特性的是（　）。

A. 有穷性　　B. 确定性　　C. 可行性　　D. 健壮性

7. 基础　关于算法的输入与输出个数，正确的是（　）。

A. 输入 0 个或多个，输出 1 个或多个　　B. 输入 1 个或多个，输出 0 个或多个

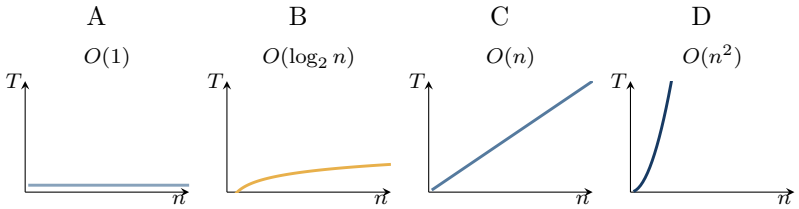
C. 输入、输出都至少 1 个　　D. 输入、输出都可以为 0 个

8. 基础　下面程序段中 `x++` 的时间复杂度为（　）。

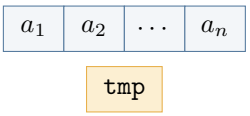
```
for(i=1; i<=n; i++)
    x++;
```

A. $O(1)$　B. $O(\log_2 n)$　C. $O(n)$　D. $O(n^2)$

四个选项对应的增长曲线如下（同一坐标范围）：



9. 基础　将长度为 n 的数组**原地**逆置（只借助一个临时变量 `tmp`），其空间复杂度为（　）。



A. $O(1)$　B. $O(\log_2 n)$　C. $O(n)$　D. $O(n^2)$

10. 基础　若某算法的执行次数 $T(n) = 2n^2 + 3n + 1$，则其时间复杂度**最紧**的大 O 表示是（　）。

A. $O(n)$　　B. $O(n^2)$　　C. $O(n^3)$　　D. $O(2^n)$

二、多项选择题（5 题，每题至少有两个正确选项）

11. 基础　下列属于**逻辑结构**的有（　）。

A. 集合　　B. 树形结构　　C. 散列存储　　D. 图形结构

12. 基础　关于顺序存储与链式存储，下列说法正确的有（　）。

A. 顺序存储中，逻辑上相邻的元素物理位置也相邻

B. 链式存储的每个结点除数据外还需存放指针

C. 链式存储可以像顺序存储一样，按序号一步算出第 i 个元素地址

D. 同一种逻辑结构可以采用不同的存储结构

13. 基础　下列描述中，**不能**称为算法的有（　）。

A. `while(1);`（永远循环，没有退出语句）

B. “加少许盐，炒到差不多就出锅”

C. 用循环求 $1 + 2 + \cdots + 100$ 并输出结果

D. 无需输入，输出 100 以内的全部素数

14. 基础　关于右表（第一行为项名），下列说法正确的有（　）。

学号	姓名	班级	成绩
100872	刘木	1442	92
100873	王一	1442	85
100874	李华	1443	78

A. 每一行是一个数据元素

B. “85”是一个数据项

C. 数据元素又称结点、记录

D. 一个数据元素只能包含一个数据项

15. 基础　关于大 O 记号，下列说法正确的有（　）。

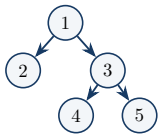
A. $3n + 5 = O(n)$　　B. $n = O(n^2)$

C. $n^2 = O(n)$　　D. $O(\log_2 n)$ 与 $O(\log_{10} n)$ 表示同一个阶

三、填空题（5 题）

16. 基础　数据类型是一组性质相同的值的集合，以及定义在这个值集合上的一组_____ 的总称。

17. 基础　树形结构中，除根结点外，每个结点有且仅有____ 个直接前驱。



18. 基础　顺序存储的线性表中，$\mathrm{LOC}(a_1)=1000$，每个元素占 2 个存储单元，则 $\mathrm{LOC}(a_{10})=$_____。

19. 基础　对笔记中的程序（右），当 $n=20$ 时，循环体 `t=t*2` 共执行____ 次，函数返回值为____。

```
t = 1;
while (t <= n) t = t * 2;
return t;
```

20. 基础　算法在输入非法数据时，能适当地作出反应或进行处理，而不会产生莫名其妙的输出结果，这一要求称为算法的_____ 性。

四、解答题（5 题）

21. 基础　某班 4 名学生按学号从小到大排成一个名单，每名学生记录“学号、姓名、班级、成绩”4 项。

(1) 名单中共有多少个数据元素、多少个数据项？

(2) 用 s_1, s_2, s_3, s_4 表示 4 名学生，写出该名单的二元组 (D, R)，并说出它是哪类逻辑结构。

s_1：学号·姓名·班级·成绩
s_2：学号·姓名·班级·成绩
s_3：学号·姓名·班级·成绩
s_4：学号·姓名·班级·成绩

22. 基础　求下面程序段中 `x++` 的执行次数 $T(n)$ 及时间复杂度。

```
for(i=1; i<=n; i++)
    for(j=1; j<=n; j++)
        x++;
```

23. 基础　按大 O 的定义证明：$5n+20=O(n)$（写出一组满足定义的 c 和 n_0）。

24. 基础　线性表 (a_1, a_2, a_3, a_4) 中每个元素占 4 个存储单元。(1) 若采用顺序存储，$\mathrm{LOC}(a_1)=100$，画出存储示意图并求 $\mathrm{LOC}(a_3)$；(2) 若采用链式存储，画出示意图，并说明此时能否用同样的公式求 a_3 的地址。

25. 基础　判断右边的程序段是否是一个算法，并说明理由。

```
int i = 1;
while (i != 10)
    i = i + 2;
```

3.2　B. 中等偏下提升训练（10 题）

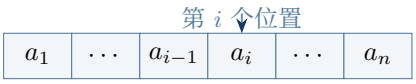
一、单项选择题（3 题）

26. 中等偏下　下面程序段（i、n 为正整数，/ 为整数除法，$n \geq 2$）中，循环体的时间复杂度为（　）。

```
i = n;
while (i > 1)
    i = i / 2;
```

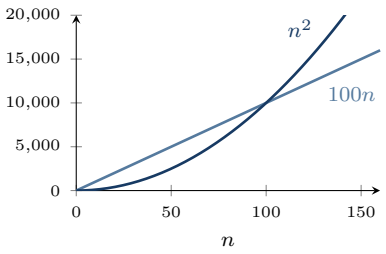
A. $O(1)$　B. $O(\log_2 n)$　C. $O(n)$　D. $O(n^2)$

27. 中等偏下　长度为 n 的顺序表，在第 i 个位置（$1 \leq i \leq n+1$）插入一个新元素，需要向后移动的元素个数为（　）。



A. $n-i$　B. $n-i+1$　C. i　D. $i-1$

28. 中等偏下　算法甲执行次数 $T_1(n) = 100n$，算法乙 $T_2(n) = n^2$（右图）。下列说法正确的是（　）。



A. 对任意 n，甲的执行次数都少于乙

B. 当 $n < 100$ 时，乙的执行次数更少

C. 两者的时间复杂度相同

D. 乙的时间复杂度更低

二、多项选择题（2 题）

29. 中等偏下　关于右边程序段，下列说法正确的有（　）。

```
for(i=1; i<=n; i++)
    for(j=1; j<=i; j++)
        x++;
```

A. x++ 共执行 $\dfrac{n(n+1)}{2}$ 次

B. 时间复杂度为 $O(n^2)$

C. 内层循环平均只执行约 $\frac{n}{2}$ 次，故时间复杂度为 $O(n)$

D. $n = 4$ 时 x++ 执行 10 次

30. 中等偏下　关于逻辑结构与存储结构，下列说法正确的有（　）。

A. 线性结构既可顺序存储，也可链式存储　　B. 树形结构只能采用链式存储

C. 存储结构是逻辑结构在计算机中的表示（映像）　　D. 逻辑结构与所用计算机无关

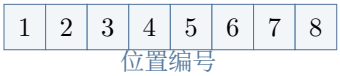
三、填空题（3 题）

31. 中等偏下　右边程序段中，当 $n = 8$ 时 count 的最终值为_____，该程序段的时间复杂度为_____。

```
count = 0;
for(i=1; i<=n; i=i*2)
    for(j=1; j<=n; j++)
        count++;
```

32. 中等偏下　某顺序表每个元素占 6 个存储单元，已知 $\mathrm{LOC}(a_3) = 130$，则 $\mathrm{LOC}(a_1) =$_____，$\mathrm{LOC}(a_8) =$_____。

33. 中等偏下　在长度为 8 的表中顺序查找（从第 1 个开始逐个比较），若要查找的元素一定在表中，且在每个位置的可能性相同，则平均比较次数为_____。



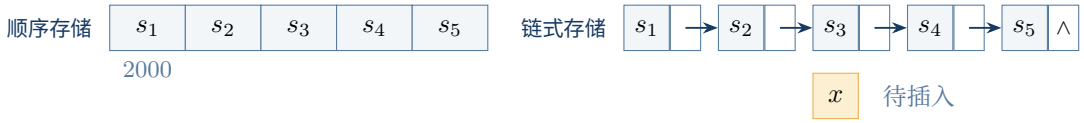
四、综合大题（2 题）

34. 中等偏下 **名单的存储与插入。**5 名学生按学号顺序组成名单 s_1, s_2, s_3, s_4, s_5，每条记录占 32 个存储单元。

(1) 写出该名单的二元组 (D, R)。

(2) 若采用顺序存储，首地址为 2000，求第 4 名学生 s_4 的起始地址。

(3) 现要把新同学 x 插到 s_2 之后。若采用顺序存储，需要移动几条记录？若采用链式存储（已找到 s_2），需要修改几个指针？



35. 中等偏下 **猜数字的两种策略。**朋友在 $1 \sim n$ 中想好一个整数，你每猜一次，他回答“大了”“小了”或“对了”。策略甲：从 1 开始依次猜 $1, 2, 3, \ldots$；策略乙：每次猜当前可能范围的中间数。已知策略乙在最坏情况下需要猜 $\lfloor \log_2 n \rfloor + 1$ 次。

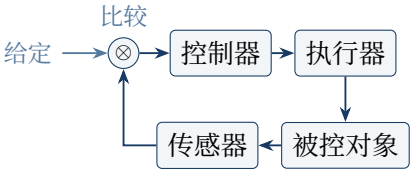
(1) 当 $n = 1000$ 时，两种策略在最坏情况下各需猜几次？

(2) 写出两种策略最坏情况的时间复杂度。

(3) 若 n 扩大到 10^6，策略乙最坏需猜几次？与 (1) 比较，你能得出什么结论？

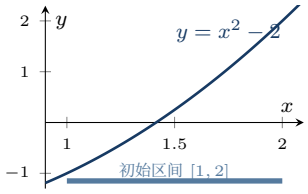
3.3　C. 跨学科迁移训练（3 题）

36. 跨学科 **（自动控制原理）**右图是一个闭环控制系统的方框图：每个方框（比较环节、控制器、执行器、被控对象、传感器）代表系统中的一个环节，箭头表示信号的传递方向。若把每个环节看作一个数据元素、把“信号从 u 传到 v”看作有序对 $\langle u, v \rangle$，则这些环节组成的结构属于（　）。



A. 集合　B. 线性结构　C. 树形结构　D. 图形结构

37. 跨学科 **（高等数学·二分法求根）**用二分法求方程 $x^2 - 2 = 0$ 在区间 $[1, 2]$ 内的根：每次取区间中点，根据中点处函数值的正负保留含根的一半，于是**每迭代一次，区间长度减半**。设精度要求为“区间长度 $\le \varepsilon$”。下列说法正确的有（　）。



A. 迭代 k 次后区间长度为 $\dfrac{1}{2^k}$

B. 要使区间长度 ≤ 0.001，至少需要迭代 10 次

C. 精度从 0.001 提高到 0.000001，所需迭代次数约增加到原来的 1000 倍

D. 所需迭代次数关于 $\dfrac{1}{\varepsilon}$ 是对数阶，即 $O\left(\log_2 \dfrac{1}{\varepsilon}\right)$

38. 跨学科 **（传感器数据采集）**某温度传感器从 $t = 0$ s 起每 0.5 s 采样一次，读数按时间先后**顺序存储**在数组中，每个读数占 4 个存储单元，$t = 0$ s 的读数存放在地址 1000 处。则 $t = 30$ s 时刻的读数存放的地址为_____。



第四章　答案与图解

全部答案速查

基础单选 1–10	1. B　2. D　3. A　4. C　5. A　6. D　7. A　8. C　9. A　10. B
基础多选 11–15	11. ABD　12. ABD　13. AB　14. ABC　15. ABD
基础填空 16–20	16. 操作　17. 1　18. 1018　19. 5；32　20. 健壮
基础解答 21–25	21. 4 个元素、16 个数据项；$D=\{s_1,s_2,s_3,s_4\}$，$R=\{\langle s_1,s_2\rangle,\langle s_2,s_3\rangle,\langle s_3,s_4\rangle\}$，线性结构　22. $T(n)=n^2$，$O(n^2)$　23. 如 $c=25$，$n_0=1$　24. $\mathrm{LOC}(a_3)=108$；链式存储不能用该公式　25. 不是算法（违反有穷性）
提升单选 26–28	26. B　27. B　28. B
提升多选 29–30	29. ABD　30. ACD
提升填空 31–33	31. 32；$O(n\log_2 n)$　32. 118；160　33. 4.5
提升大题 34–35	34. (1) $D=\{s_1,\ldots,s_5\}$，$R=\{\langle s_i,s_{i+1}\rangle \mid i=1,2,3,4\}$；(2) 2096；(3) 移动 3 条，改 2 个指针　35. (1) 甲 1000 次、乙 10 次；(2) $O(n)$、$O(\log_2 n)$；(3) 20 次
跨学科 36–38	36. D　37. ABD　38. 1240

4.1　基础巩固 · 详细解析

单项选择题

第 1 题　基础　答案：**B**　考查：K1 数据元素是基本单位

选项 A、B、D 都能在成绩表上“圈”出来，C 是“值 + 操作”，不是表上的某一块。三幅小图用同一张表，只改变圈出的范围：

学号	姓名	班级	成绩
100872	刘木	1442	92
100873	王一	1442	85
100874	李华	1443	78

A 数据项：一格 ✗ 错误　B 数据元素：一行 ✓ 正确　D 数据对象：整张表 ✗ 错误

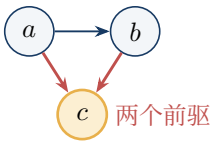
【逐项】 A：一格是数据项，它是最小单位，不是基本单位。B：一行是一名学生的完整记录，程序增删查时把它当作一个整体，是**基本单位**。C：数据类型是“值集合 + 操作”的总称，根本不是“单位”（笔记原写法的错误正在这里）。D：整张表是同类元素的集合，范围太大。

【易错】 “基本”与“最小”混淆。

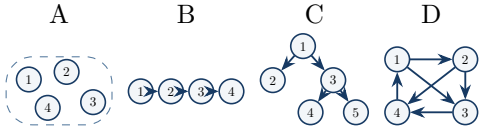
【一句话】 元素是基本，数据项最小。

第 2 题　基础　答案：**D**　考查：K2 由 (D,R) 画图；K4 树与图的区别

【思路】 先“画点、连箭头”，再数每个点的前驱个数。三条有序对画出右图：c 同时有 a、b 两个直接前驱（红色），这是判断的关键证据。



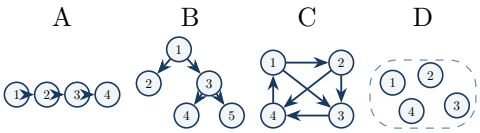
【逐项】 A 集合：要求没有箭头，而这里有 3 条 ✗ 错误。B 线性：要求每个元素至多一个前驱，c 有两个 ✗ 错误。C 树形：除根外每个结点恰有一个前驱，c 违反 ✗ 错误。D 图形：多对多，允许一个元素有多个前驱 ✓ 正确。



上排四幅为 A–D 四种结构的“标准形状”。本题图与 C 的区别正是“一个结点能否有两个前驱”。

【一句话】 多了一个前驱，树就变成了图。

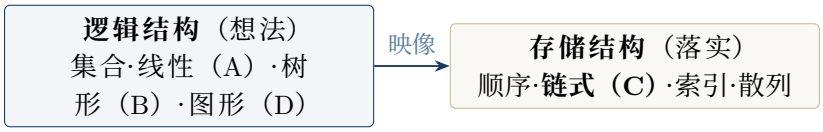
第 3 题 基础　答案：**A**　考查：K4 线性结构



A 对应线性结构：每个点一进一出 ✓ **正确**。B 一对多是树形结构（根 1 有两个后继）✗ **错误**。C 多对多是图形结构 ✗ **错误**。D 是集合 ✗ **错误**。

【一句话】 线性 = 排成一队 = 一对一。

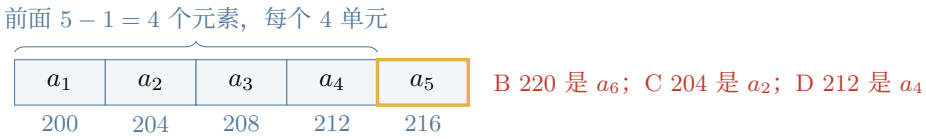
第 4 题 基础　答案：**C**　考查：K4 与 K5 的区分



A、B、D 描述元素之间“逻辑上”是什么关系，与计算机无关；只有 C“链式”说的是关系在内存中怎样表示（用指针）。

【易错】 看到“结构”二字就认为同类。判断方法：问一句“它跟内存有没有关系”。

第 5 题 基础　答案：**A**　考查：K5 顺序存储地址公式

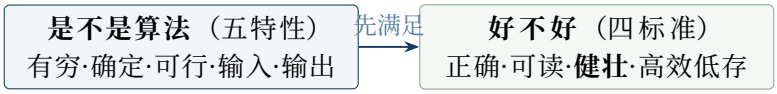


$\mathrm{LOC}(a_5) = 200 + (5-1) \times 4 = 216$。错误选项都对应图中另一个格子：B 把 $i-1$ 写成了 i（$200+5\times4$），D 少跳一格。

【易错】 用 i 而不是 $i-1$。

第 6 题 基础　答案：**D**　考查：K6 五个特性与 K7 好算法标准的区分

健壮性属于“好算法的标准”（第 2.2 节），不是“是不是算法”的特性。五个特性是：有穷、确定、可行、输入、输出。A、B、C 都在其中。



【一句话】 健壮性是“好”的标准，不是“是”的特性。

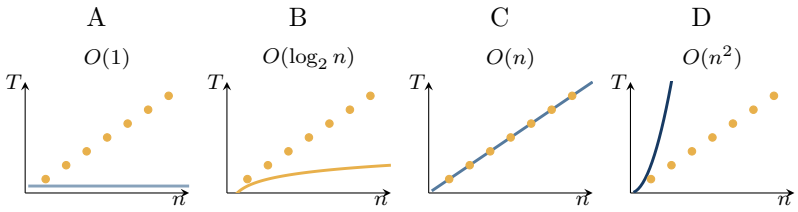
第 7 题 基础　答案：**A**　考查：K6 输入、输出个数

输入可以为 0（如“输出 100 以内的素数”不需要外部数据），输出至少 1（没有输出就没有解决任何问题）。B 把两者说反；C 错在输入不必至少 1；D 错在输出不能为 0。本题四个选项都是数量判断，不具备图形意义，不配选项图。

【一句话】 输入可无，输出必有。

第 8 题　基础　答案：**C**　考查：K8 单层循环

循环变量 i 从 1 走到 n，每次加 1，`x++` 执行 n 次，$T(n)=n$。下图在四个选项的曲线上叠加了程序实际执行次数（黄点，$n=2,4,\dots,14$）：

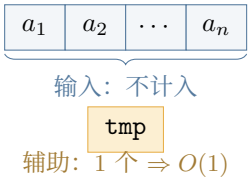


只有 C 的曲线与黄点重合。A 的水平线越来越低于黄点（n 次不是常数）；B 的对数曲线增长太慢；D 的抛物线增长太快、很快冲出图框。

【易错】 把“只有一行代码”误认为 $O(1)$——看的是执行次数，不是代码行数。

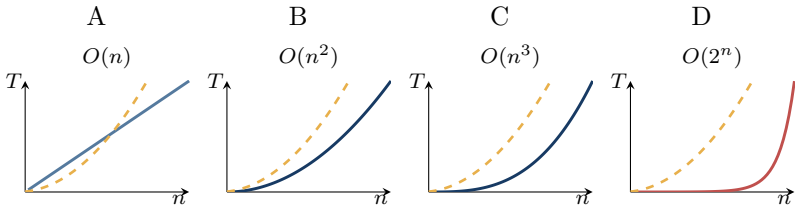
第 9 题　基础　答案：**A**　考查：K9 空间复杂度只算辅助空间

无论 n 多大，额外只用了 1 个 `tmp`（右图黄格），辅助空间是常数，$S(n)=O(1)$。数组本身的 n 个单元是输入数据，**不计入**。C 是把输入空间也算进去的错误结果；B、D 没有任何依据。



【易错】 “数组有 n 个元素，所以空间 $O(n)$”。

第 10 题　基础　答案：**B**　考查：K8 取最高阶项；大 O 是上界（对照：高数中“同阶”）

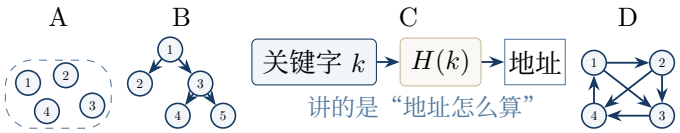


黄色虚线为 $T(n)$（四幅图用同一缩放比例画出）。A：直线迟早被 $T(n)$ 超过，不是上界 ✗ **错误**。B：抛物线与 $T(n)$ 形状一致，同阶，最紧 ✓ **正确**。C：n^3 最终在 $T(n)$ 上方，严格说 $T(n)=O(n^3)$ 也成立，但不是最紧的 ✗ **错误**。D 同理，更松 ✗ **错误**。

【对照说明】 这正是 K8 对照表中 △ 的一条：“$f=O(g)$ 不等于 f 与 g 同阶”。题目问“最紧”，所以选 B。

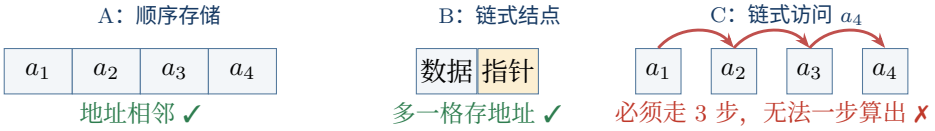
多项选择题

第 11 题　基础　答案：**ABD**　考查：K4、K5 区分



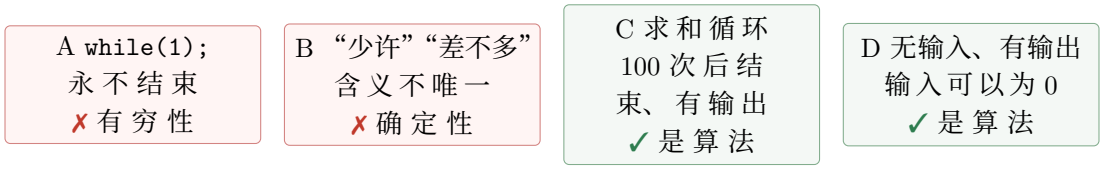
A 集合、B 树形、D 图形都是“点 + 箭头”层面的逻辑结构 ✓ **正确**。C 散列存储描述的是“由关键字算出存储地址”，属于存储结构 ✗ **错误**——它的图里出现了“地址”，这是存储结构的标志。

第 12 题　基础　答案：**ABD**　考查：K5 顺序与链式



A 是顺序存储的定义 ✓ **正确**。B 链式结点 = 数据域 + 指针域 ✓ **正确**。C 链表中各结点地址无规律，只能从头沿指针走 $i-1$ 步，不能随机存取 ✗ **错误**（对照 K5：“递推数列也能直接写出第 i 项” ✗）。D 线性结构可顺序也可链式 ✓ **正确**（图见第 1.5 节易错点）。

第 13 题　基础　答案：**AB**　考查：K6 五特性（本题选“不是算法”的）



A 违反有穷性；B 违反确定性；C 满足全部五特性；D 输入为 0 是允许的。所以“不能称为算法”的是 A、B。

【易错】 把 D 误选：输入 0 个是合法的。

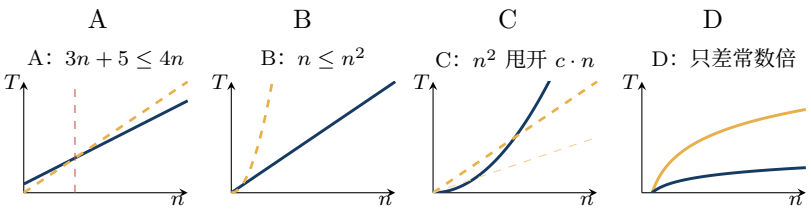
第 14 题　基础　答案：**ABC**　考查：K1

学号	姓名	班级	成绩
100872	刘木	1442	92
100873	王一	1442	85
100874	李华	1443	78

A：一行 = 一个数据元素　B：“85” = 一个数据项

A ✓ **正确**；B “85”是王一的成绩，一个独立含义的格子 ✓ **正确**；C 是教材中的别称 ✓ **正确**；D 每行有 4 个数据项，“若干个数据项构成一个数据元素” ✗ **错误**。

第 15 题　基础　答案：**ABD**　考查：K8 大 O 定义；对照：换底公式

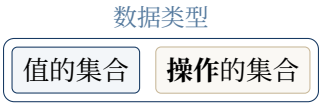


实线为左边函数，黄色虚线为 $c \cdot f(n)$（为便于比较，各图纵向统一缩放）。A：取 $c=4, n_0=5$，$3n+5 \le 4n \iff n \ge 5$ ✓ **正确**。B：$n \ge 1$ 时 $n \le n^2$，取 $c=1$ ✓ **正确**（上界不必紧）。

C：若 $n^2 \le cn$，则 $n \le c$，对足够大的 n 不可能，图中抛物线终将越过任何一条直线 ✗ **错误**。D：$\log_2 n = \log_{10} n / \log_{10} 2 \approx 3.32 \log_{10} n$，两条曲线形状相同、只差常数倍 ✓ **正确**。

填空题

第 16 题　基础　答案：**操作**　考查：K3（纠正笔记原写法）

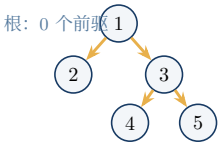


数据类型 = 值集合 + 定义在其上的一组操作。

【易错】 填“数据项”“单位”——这正是笔记原句的混淆。

第 17 题　基础　答案：**1**　考查：K4 树形结构

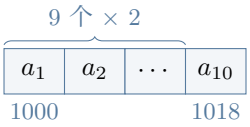
在题图中逐一检查：结点 2、3、4、5 都恰有一条进入的箭头（右图黄色），只有根 1 没有。若某结点有两个前驱，就成了图形结构（见第 2 题）。



【一句话】 树：一个爹，可以多个娃。

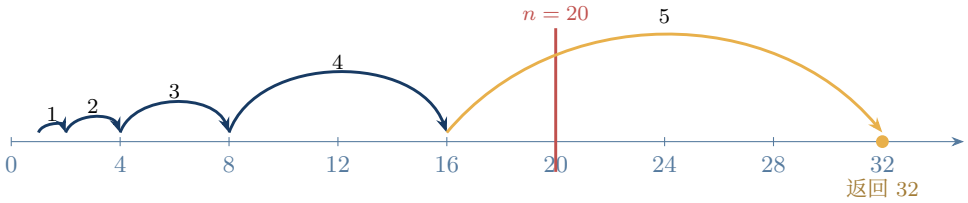
第 18 题　基础　答案：**1018**　考查：K5 地址公式

$$\mathrm{LOC}(a_{10}) = 1000 + (10-1) \times 2 = 1018。$$



【易错】 算成 $1000 + 10 \times 2 = 1020$。

第 19 题　基础　答案：**5；32**　考查：K8 笔记例题的精确次数



t：$1 \to 2 \to 4 \to 8 \to 16 \to 32$，前 5 次判断 $t \le 20$ 都成立，共执行 5 次；第 6 次判断 $32 \le 20$ 不成立。验算公式：$\lfloor \log_2 20 \rfloor + 1 = 4 + 1 = 5$。返回值 32 是大于 20 的最小 2 的幂。

【易错】 答 $\log_2 20 \approx 4.32$ 次——次数必须是整数。

第 20 题　基础　答案：**健壮**　考查：K7

非法输入 → 适当反应，而不是崩溃或乱输出，即健壮性（鲁棒性）。区分：“正确性”只针对合法输入。本题为纯术语，无需配图。

解答题

第 21 题　基础　考查：K1 + K2 + K4

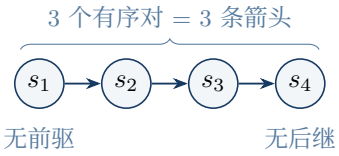
【题意】 名单 = 4 个元素按学号排成一队。

【(1)】 一名学生的完整记录是一个数据元素，共 **4 个**；每个元素有 4 个数据项，共 $4 \times 4 = \mathbf{16}$ **个**数据项。

【(2)】 $D = \{s_1, s_2, s_3, s_4\}$；“按学号先后”相邻两人构成有序对：

$$R = \{\langle s_1, s_2\rangle, \langle s_2, s_3\rangle, \langle s_3, s_4\rangle\}.$$

画出后（右图）：s_1 无前驱，s_4 无后继，其余各一个前驱一个后继，为**线性结构**。



【检查】 有序对个数 = 元素数 $-1 = 3$，线性结构总是如此。

【易错】 写成 $\{s_1, s_2\}$（无序），或漏写方向。

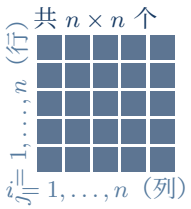
第 22 题　基础　考查：K8 双重循环、乘法规则

【基本语句】 `x++`。

【第 1 步】 外层 i 取 $1, \ldots, n$，共 n 次。

【第 2 步】 每次外层，内层 j 取 $1, \ldots, n$，`x++` 执行 n 次，与 i 无关。

【第 3 步】 $T(n) = \underbrace{n + n + \cdots + n}_{n\text{ 个}} = n^2$，对应右图 $n \times n$ 的方阵（$n = 5$）中的方块数。



【结果】 $T(n) = n^2$，时间复杂度 $O(n^2)$。

【检查】 $n = 2$ 时手算：$(1,1), (1,2), (2,1), (2,2)$ 共 4 次 $= 2^2$。

【同类题】 两层独立循环 $\Rightarrow$ 次数相乘。

第 23 题　基础　考查：K8 大 O 定义

【要证】 存在 $c > 0, n_0$，使 $n \geq n_0$ 时 $5n + 20 \leq cn$。

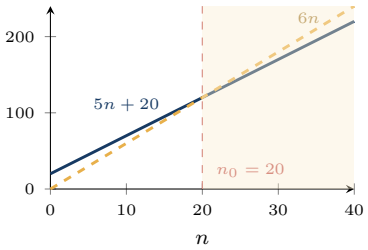
【思路】 把常数 20 放大成 $20n$。

【证明】 当 $n \geq 1$ 时 $20 \leq 20n$（因为 $n \geq 1$），所以

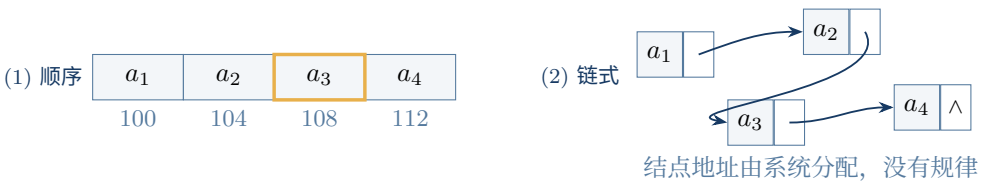
$$5n + 20 \leq 5n + 20n = 25n.$$

取 $c = 25$，$n_0 = 1$，定义成立，故 $5n + 20 = O(n)$。

【另一组】 取 $c = 6$：$5n + 20 \leq 6n \iff n \geq 20$，即 $n_0 = 20$（右图交点）。两组都对，c, n_0 不唯一。



第 24 题　基础　考查：K5 两种存储的画法与地址计算

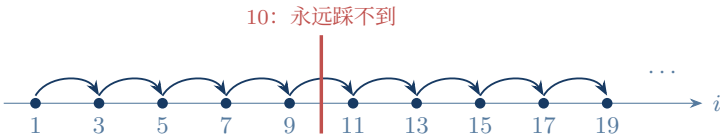


【(1)】 $\mathrm{LOC}(a_3) = 100 + (3 - 1) \times 4 = \mathbf{108}$。

【(2)】 **不能**。公式成立的前提是“元素连续存放、大小相同”；链式存储中结点位置任意，只能从第一个结点出发，沿指针走 2 步才能找到 a_3。

【易错】 以为链表结点也是一个挨一个排的。

第 25 题 基础　考查：K6 有穷性的判断



【分析】 i 从 1 开始每次加 2，依次为 $1,3,5,7,9,11,\ldots$，始终是奇数（奇数 + 偶数 = 奇数），而 10 是偶数，所以条件 `i!=10` 永远成立，循环永不结束。

【结论】 **不是算法**，违反**有穷性**。

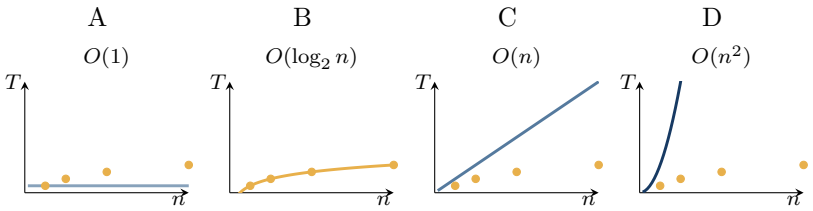
【改法】 把条件改为 `i<10`，则执行到 $i=11$ 时停止。

【易错】 只看到“i 在增大”就以为一定会停——要检查“终止条件能否真的被满足”。

4.2　中等偏下提升 · 详细解析

第 26 题 中等偏下　答案：**B**　考查：K8 折半型循环（笔记例题的“逆过程”）

知识链： 写出 i 的变化 → 求次数 → 取阶。i：$n \to \lfloor n/2 \rfloor \to \lfloor n/4 \rfloor \to \cdots$，每次至少减半。设执行 k 次后 $i \le 1$ 停止，则约 $n/2^k \le 1$，$k \approx \log_2 n$；精确地，次数为 $\lfloor \log_2 n \rfloor$（如 $n=16$：$16 \to 8 \to 4 \to 2 \to 1$，4 次）。下图在四个选项曲线上叠加实际次数（黄点）：

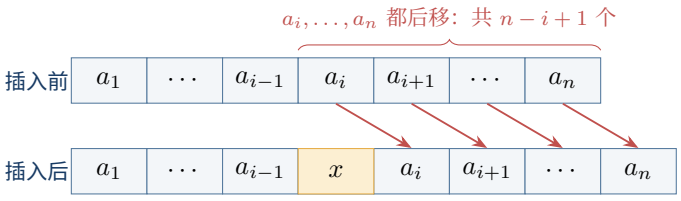


黄点 $(2,1),(4,2),(8,3),(16,4)$ 恰好落在 B 的对数曲线上；A 的水平线被黄点超出；C、D 增长远快于黄点。

【对照】 与笔记例题“t 从 1 翻倍到 n”方向相反、次数同阶。

【易错】 看到 `while` 和 n 就选 $O(n)$。

第 27 题 中等偏下　答案：**B**　考查：K5 顺序存储的插入代价



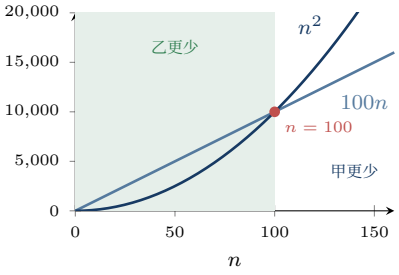
从 a_i 到 a_n 共 $n-i+1$ 个元素都要后移一格（必须从 a_n 开始倒着移，否则会覆盖）。验算边界：$i=n+1$（插在表尾）移动 0 个 ✓；$i=1$（插在表头）移动 n 个 ✓。

【逐项】 A $n-i$：漏了 a_i 本身，$i=1$ 时得 $n-1$ 与实际不符。C i、D $i-1$：数的是前面的元素，而前面的元素不动。

【检查法】 用两个端点 $i=1$、$i=n+1$ 代入，最快排除错误选项。

第 28 题　中等偏下　答案：**B**　考查：K8 大 O 只描述“n 足够大”时的趋势；对照：高数中比较阶

【思路】 先求两曲线交点：$100n = n^2 \iff n = 100$（$n > 0$）。



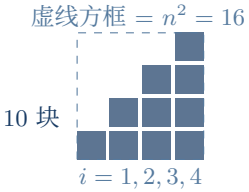
【逐项】 A：$n < 100$ 时 $n^2 < 100n$，乙反而更少 ✗ **错误**。B：例如 $n = 10$，甲 1000 次，乙 100 次 ✓ **正确**（右图阴影区）。C：甲 $O(n)$、乙 $O(n^2)$，不同 ✗ **错误**。D：乙的阶更高 ✗ **错误**。

【对照说明】 高数中“$\lim_{n\to\infty} \frac{100n}{n^2} = 0$，所以 $100n$ 是比 n^2 低阶的无穷大”——这句话只说 $n \to \infty$，对 $n < 100$ 不作任何保证。

第 29 题　中等偏下　答案：**ABD**　考查：K8 内层依赖外层的双重循环

【第 1 步】 外层第 i 次时，内层 $j = 1, \dots, i$，`x++` 执行 i 次。

【第 2 步】 $T(n) = 1 + 2 + \cdots + n = \dfrac{n(n+1)}{2}$，对应右图的“阶梯”（$n = 4$ 时共 10 块）。

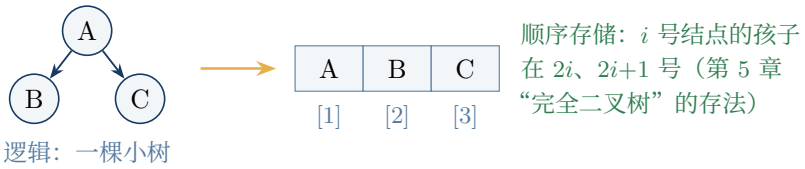


【第 3 步】 $\dfrac{n(n+1)}{2} = \dfrac{1}{2}n^2 + \dfrac{1}{2}n$，最高阶 n^2，故 $O(n^2)$。

【逐项】 A 由第 2 步 ✓ **正确**。B 由第 3 步 ✓ **正确**。C：“平均约 $\frac{n}{2}$ 次”没错，但外层还有 n 次，总次数约 $n \cdot \frac{n}{2} = \frac{n^2}{2}$，系数 $\frac{1}{2}$ 不改变阶，仍是 $O(n^2)$ ✗ **错误**——阶梯面积是方框的一半，形状仍是“平方级”。D：$\frac{4\times5}{2} = 10$ ✓ **正确**。

【易错】 C 忘了乘外层次数。

第 30 题　中等偏下　答案：**ACD**　考查：K4 与 K5 的关系



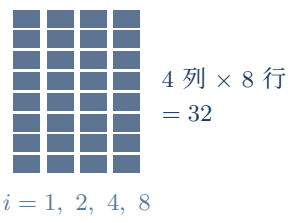
A：线性表既有顺序表也有链表 ✓ **正确**。B：树也可以顺序存储，上图按层次把结点放进数组，结点 i 的左右孩子位于 $2i$、$2i+1$ 号位置，关系由“位置的计算规则”体现 ✗ **错误**（本例为**拓展联想**，第 5 章详讲）。C：存储结构就是逻辑结构在计算机中的表示 ✓ **正确**。D：逻辑结构只描述关系，与计算机无关 ✓ **正确**。

第 31 题　中等偏下　答案：**32；$O(n \log_2 n)$**　考查：K8 乘法规则 + 对数型循环

【外层】 $i = 1, 2, 4, 8$（≤ 8），共 4 次 $= \lfloor \log_2 8 \rfloor + 1$。

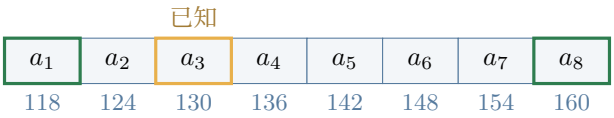
【内层】 每次外层都执行 $n = 8$ 次，与 i 无关。

【合计】 $4 \times 8 = 32$。一般地 $T(n) = n(\lfloor \log_2 n \rfloor + 1)$，由乘法规则得 $O(n) \cdot O(\log_2 n) = O(n \log_2 n)$。



【易错】 把外层也当作 n 次，得 64 与 $O(n^2)$。

第 32 题　中等偏下　答案：**118；160**　考查：K5 地址公式的逆用（对照：等差数列）



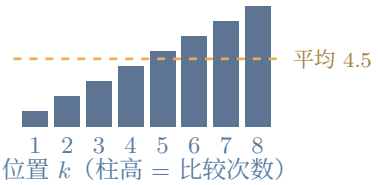
$\mathrm{LOC}(a_3) = \mathrm{LOC}(a_1) + 2 \times 6 \Rightarrow \mathrm{LOC}(a_1) = 130 - 12 = 118$；$\mathrm{LOC}(a_8) = 118 + 7 \times 6 = 160$（或 $130 + 5 \times 6 = 160$）。

【对照】 正是“等差数列已知第 3 项和公差，求首项与第 8 项”。

第 33 题　中等偏下　答案：**4.5**　考查：K8 平均情况

在第 k 个位置找到需比较 k 次，各位置概率均为 $\frac{1}{8}$：

$$\frac{1+2+\cdots+8}{8} = \frac{36}{8} = 4.5.$$



一般地为 $\frac{n+1}{2}$，最坏 n 次，复杂度 $O(n)$。

【易错】 答 4 ($\frac{n}{2}$)，漏了“比较次数从 1 而不是从 0 数起”。

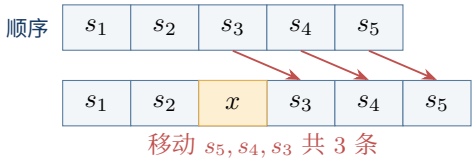
第 34 题　中等偏下　考查：K1、K2、K5 综合（知识链：结构 → 地址 → 操作代价）

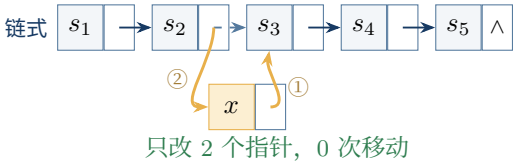
【题意】 同一份名单，先用 (D, R) 描述，再比较两种存储方式下“找”和“插”的代价。

【(1)】 $D = \{s_1, s_2, s_3, s_4, s_5\}$，$R = \{\langle s_1, s_2\rangle, \langle s_2, s_3\rangle, \langle s_3, s_4\rangle, \langle s_4, s_5\rangle\}$，线性结构。

【(2) 为什么用地址公式】 顺序存储满足“连续、等长”。$\mathrm{LOC}(s_4) = 2000 + (4-1) \times 32 = \mathbf{2096}$。

【(3)】 插到 s_2 之后，即成为第 3 条：





顺序存储：s_3, s_4, s_5 依次后移，**移动 3 条**（与第 27 题公式一致：$n - i + 1 = 5 - 3 + 1 = 3$）。链式存储：**修改 2 个指针**——①先令 `x->next = s2->next`（x 指向 s_3），②再令 `s2->next = x`。

【为什么顺序不能颠倒】 若先做②，s_2 原来指向 s_3 的指针就被覆盖，s_3 之后的结点全部“丢失”。

【合理性检查】 顺序存储找第 4 条一步算出、插入要搬家；链式存储插入只改指针、但找第 4 条要走 3 步——两者各有所长，与第 1.5 节对比表一致。

【变式思考】 若改为删除 s_2，两种存储各需移动几条记录 / 修改几个指针？（顺序：移动 3 条；链式：改 1 个指针。）

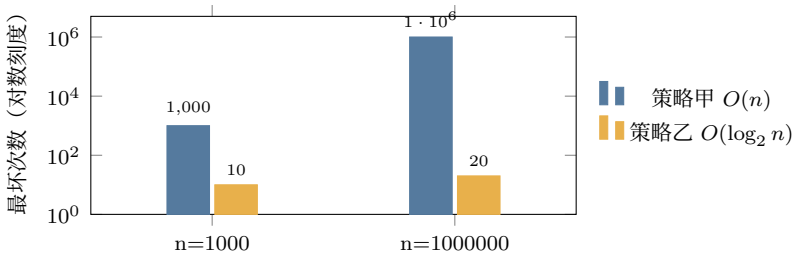
第 35 题　中等偏下　考查：K8 线性阶与对数阶的实际差距（对照：生活中的猜数字）

【题意】 比较“逐个试”与“每次砍一半”两种策略在最坏情况下的次数。

【(1)】 甲：答案为 1000 时要从 1 猜到 1000，**1000 次**。乙：$\lfloor \log_2 1000 \rfloor + 1 = 9 + 1 = \mathbf{10}$ **次**（因为 $2^9 = 512 \le 1000 < 1024 = 2^{10}$）。

【(2)】 甲 $T(n) = n$，$O(n)$；乙 $O(\log_2 n)$。

【(3)】 $2^{19} = 524288 \le 10^6 < 2^{20}$，所以 $\lfloor \log_2 10^6 \rfloor = 19$，乙最坏 **20 次**。



【结论】 n 扩大 1000 倍，甲的次数也扩大 1000 倍，而乙只多了 10 次。对数阶算法的威力在于：**规模每翻一倍，代价只加 1**。

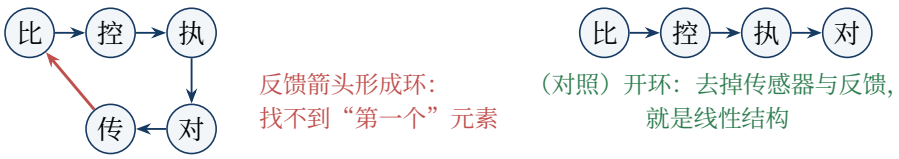
【易卡点】 $\lfloor \log_2 1000 \rfloor$ 的计算：找出“不超过 1000 的最大 2 的幂”的指数。

【变式】 若 $n = 2^{30}$（约 10 亿），乙最坏几次？（31 次。）

4.3　跨学科迁移 · 详细解析

第 36 题　跨学科　答案：**D**

当前专题知识： K2 由有序对画结构、K4 结构类型判定。**借用的外部情境：** 自动控制原理中的闭环方框图。**为什么不算超纲：** 题干已说明“方框 = 环节、箭头 = 信号方向”，不需要任何控制理论知识。**迁移点：** 方框图本身就是“点 + 箭头”，与 (D, R) 的图示完全同构。

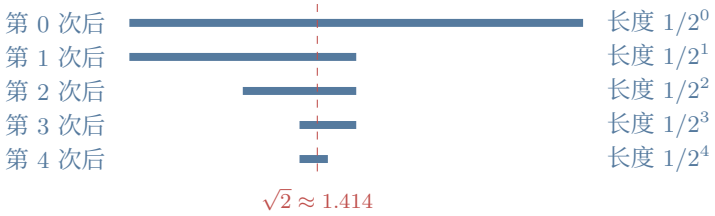


【逐项】 A：有箭头，不是集合 ✗ **错误**。B：每个环节确实“一进一出”，但反馈形成环，没有无前驱的第一个元素，不满足线性结构定义（正是第 1.4 节的“环形反例”）✗ **错误**。C：树必须有唯一的根且无环 ✗ **错误**。D：图形结构允许环 ✓ **正确**。

【对照说明】 控制课上“开环 / 闭环”的区别，在数据结构的眼光下恰好是“线性结构 / 有环的图形结构”的区别。

第 37 题　跨学科　答案：**ABD**

当前专题知识：K8 对数阶、“每次减半”与笔记例题同构。**借用的外部情境：**高等数学 / 数值计算中的二分法求根。**为什么不算超纲：**题干已给出“每次区间长度减半”这一全部所需事实，不需要证明二分法的正确性。**迁移点：**“区间长度 $\frac{1}{2^k} \le \varepsilon$”与“$2^k > n$”是同一个不等式。



【逐项】　A：长度从 1 开始每次减半，k 次后为 $\frac{1}{2^k}$ ✓ **正确**（图中各行）。

B：需 $\frac{1}{2^k} \le 0.001 \iff 2^k \ge 1000$。$2^9 = 512 < 1000$，$2^{10} = 1024 \ge 1000$，所以至少 10 次 ✓ **正确**。

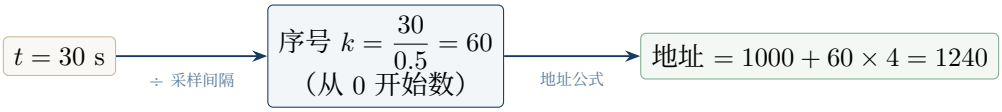
C：$\varepsilon = 10^{-6}$ 时需 $2^k \ge 10^6$，$2^{19} < 10^6 \le 2^{20}$，需 20 次，只是 10 次的 **2 倍**，而不是 1000 倍 ✗ **错误**。

D：$k = \lceil \log_2 \frac{1}{\varepsilon} \rceil$，对 $\frac{1}{\varepsilon}$ 是对数阶 ✓ **正确**。

【对照说明】　C 的错误正是把“$\frac{1}{\varepsilon}$ 扩大 1000 倍”误当成“次数扩大 1000 倍”，即用线性阶的直觉去套对数阶——与第 35 题“猜数字”的结论完全一致。

第 38 题　跨学科　答案：**1240**

当前专题知识：K5 顺序存储地址公式。**借用的外部情境：**传感器等间隔采样（测控技术）。**为什么不算超纲：**只需理解“每 0.5 s 一个读数、按时间顺序存放”，题干已给出。**迁移点：**“时间 t”先换算成“第几个读数”，再套地址公式。



$t = 0$ s 是第 0 个读数（相当于 a_1），$t = 30$ s 是第 60 个，即 a_{61}：$\mathrm{LOC}(a_{61}) = 1000 + (61-1) \times 4 = 1240$。

【易错】　写成 $1000 + 30 \times 4 = 1120$（把秒数直接当序号）或 $1000 + 61 \times 4 = 1244$（多跳一格）。两种错误都可以用 $t = 0.5$ s 验算：它应在 1004。

第五章　总结与延伸

高频易错点总表

错误想法	正确想法	相关题
数据元素是最小单位	数据元素是基本单位，数据项是最小单位	1, 14
数据的不可分割单位是数据类型（笔记原句）	数据类型 = 值集合 + 操作；不可分割的是数据项	16
(D, R) 是三元组；D 是“收集数据”	二元组；D 是数据元素的有限集	例 1.2
$\langle a, b\rangle$ 与 $\langle b, a\rangle$ 相同	有序对有方向，画箭头 $a \to b$	21
一对多 = 一个结点可有多个前驱	树中除根外每个结点恰有一个前驱	2, 17
环形结构是线性结构	线性结构必须有唯一的“第一个”和“最后一个”	36
链式结构是逻辑结构	顺序、链式、索引、散列都是存储结构	4, 11
线性结构只能顺序存储；树只能链式存储	同一逻辑结构可用不同存储结构	12, 30
链表也能一步算出第 i 个地址	只有顺序存储能随机存取	12, 24
能运行的程序就是算法	必须满足有穷性等五个特性	13, 25
健壮性是算法五特性之一	健壮性是“好算法”的标准	6
写成 $O(2n)$、$O(n^2+n)$	去系数、去低阶项：$O(n)$、$O(n^2)$	10
$O(n^2)$ 的算法在任何时候都更慢	大 O 只描述 n 足够大时的趋势	28
$\log_2 n$ 就是精确次数	笔记例题精确次数为 $\lfloor \log_2 n \rfloor + 1$	19
内层平均 $\frac{n}{2}$ 次，所以总共 $O(n)$	还要乘外层 n 次，得 $O(n^2)$	29
空间复杂度要算上输入数组	只算辅助空间	9

基础解题步骤总结

四类基础题的通用步骤

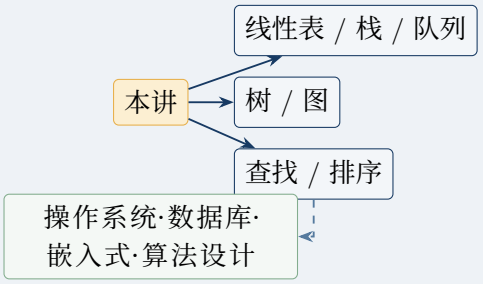
①判断逻辑结构： 画点 → 按有序对连箭头 → 数每个点的前驱、后继个数 → 查判定树（图 9）。
②顺序存储求地址： 确认“连续、等长” → 数清目标前面有几个元素 $(i-1)$ → $\mathrm{LOC}(a_1)+(i-1)L$ → 用相邻元素验算。
③判断是否为算法： 逐条核对有穷、确定、可行、输入（≥ 0）、输出（≥ 1）；重点检查终止条件能否真正满足。
④求时间复杂度： 找基本语句 → 写循环变量的变化规律 → 算次数 $T(n)$（独立嵌套相乘、先后相接取大）→ 去系数、去低阶 → 写 $O(\cdot)$；必要时代入小 n 验算。

后续课程连接

本讲知识以后在哪里出现

本课程：第 2 章线性表（顺序表、链表，直接用 K5 的两种存储与插入代价）；第 3 章栈与队列；第 5 章树与二叉树（完全二叉树的顺序存储即第 30 题 B 的拓展）；第 6 章图（邻接矩阵 = 关系矩阵）；第 7 章查找（折半查找 $O(\log_2 n)$、散列表）；第 8 章排序（$O(n^2)$ 与 $O(n\log_2 n)$ 的对比）。

后续课程：操作系统（进程队列、内存管理中的链式分配）、数据库（索引存储、B 树）、计算机控制系统与嵌入式开发（采样数据缓冲区、实时算法的时间预算）、算法设计与分析（Θ,Ω 记号、递推式）。



相近概念总对比

	数据项	数据元素	数据类型
本质	最小单位（一格）	基本单位（一行）	值集合 + 操作
例子	成绩“92”	刘木的整条记录	C++ 的 `int`
	逻辑结构	存储结构	算法
回答的问题	元素间是什么关系	关系在内存里怎样表示	操作怎样一步步完成
与计算机有关吗	无关	有关	描述与语言无关，执行代价与存储结构有关
种类	集合、线性、树形、图形	顺序、链式、索引、散列	——
	正确性	健壮性	有穷性
针对	合法输入的结果	非法输入的反应	能否在有限步内结束
属于	好算法标准	好算法标准	算法的必要特性

跨学科 / 相似知识总对比表

本讲知识 ↔ 相似知识	类型	最关键的“异”与迁移警示	位置
数据元素 / 数据项 ↔ C++ 结构体 / 成员	思想重现	成员可以再分（`string`、数组），不一定是数据项；`sizeof` 有对齐填充 ✗	K1
(D, R) ↔ 离散数学的二元关系、有向图	推广	同一张图两种读法；R 只含“直接”关系，“间接在前”需求传递闭包 △	K2
C++ `int` ↔ 整数集 $\mathbb{Z}$	同名异义	`int` 有限，会溢出；封闭性 ✗、$(a/b)\cdot b = a$ ✗	K3
逻辑结构 ↔ 图论中的图、树	推广	线性 $\subset$ 树 $\subset$ 图；树的遍历用于图须标记已访问 △	K4
顺序 / 链式存储 ↔ 数列通项 / 递推	类比同构	地址公式即等差数列通项；C++ 下标从 0 起时公式要改 △	K5
算法 ↔ 数学函数	同名异义	同一函数可有代价悬殊的多个算法；“函数相等即一样好” ✗	K6
算法健壮性 ↔ 控制系统鲁棒性	同名异义	共同思想是“考虑非理想情况”；本课程健壮性为定性要求	K7
算法中的大 O ↔ 高数中无穷大的阶	思想重现	极限方向相反：$n \to \infty$ 保留高次项；“丢高次项” ✗；O 只是上界 △	K8
时间复杂度 ↔ 二分法迭代次数	类比同构	区间减半与 t 翻倍同一个不等式；精度提高 1000 倍，次数只加约 10	第 37 题
开环 / 闭环控制 ↔ 线性 / 有环图形结构	类比同构	反馈箭头使“第一个元素”消失	第 36 题

参考资料

原始教学依据

- 课堂手写笔记两页（Memo No. 529–530，2026 年 10 月 13 日），本讲义第“原始资料重新编码”节逐条列出。

核心定义与公式核验

- 严蔚敏、吴伟民.《数据结构（C 语言版）》. 清华大学出版社. 第 1 章“绪论”（数据、数据元素、数据项、数据结构二元组、逻辑结构与存储结构、ADT、算法五特性、好算法标准、时间与空间复杂度）。
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. Introduction to Algorithms, 3rd ed., MIT Press, 2009. 第 3 章（渐近记号 O, Ω, Θ 的定义）。
- NIST, Dictionary of Algorithms and Data Structures: “big-O notation”. `https://xlinux.nist.gov/dads/HTML/bigOnotation.html`

学科史资料

- D. E. Knuth. Big Omicron and big Omega and big Theta. ACM SIGACT News, 8(2), 1976.（O 记号源于 Bachmann 1894、经 Landau 推广，及 Knuth 倡导统一使用的论述）
- J. Hartmanis, R. E. Stearns. On the computational complexity of algorithms. Trans. AMS, 117: 285–306, 1965. `https://doi.org/10.1090/S0002-9947-1965-0170805-7`
- D. E. Knuth. The Art of Computer Programming, Vol. 1: Fundamental Algorithms. Addison-Wesley, 1968.
- N. Wirth. Algorithms + Data Structures = Programs. Prentice-Hall, 1976.
- 链表与 IPL 语言（Newell、Shaw、Simon，1955–1956）：Wikipedia “Linked list”“Information Processing Language”词条及其所引原始文献。

最后的自检

如果你现在能不看讲义说清楚：“一张成绩表的行和格各是什么；(D, R) 怎么画；四种逻辑结构各几对几；顺序和链式各快在哪里；什么是算法、什么是好算法；为什么笔记里那段程序是 $O(\log_2 n)$，而且精确次数是 $\lfloor \log_2 n \rfloor + 1$”——那么这一讲你已经真正学会了。